Self-Calibrating VLAs: Test-Time Embodiment Alignment for Vision-Language-Action Models

Anonymous Authors

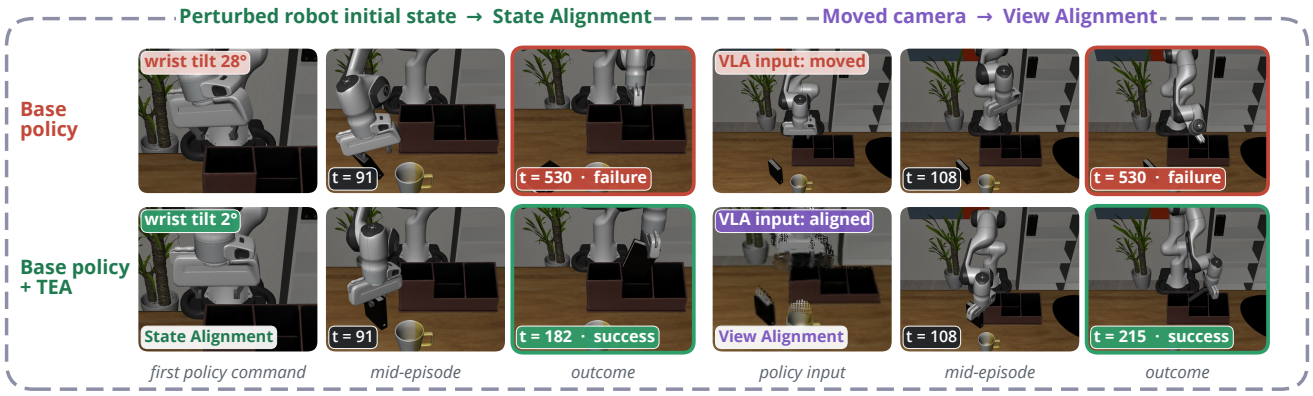

Figure 1. **Self-calibrating VLAs.** One LIBERO-Long task, *put the book in the back compartment of the caddy*, under its perturbed robot initial state (left) and its moved camera (right), with VLA-Adapter-Pro alone (top) and with TEA (bottom) on identical episodes. State Alignment keeps the wrist at the training start (2° instead of 28° at the first command); View Alignment hands the policy the view from its training camera. The base policy fails both; with TEA, the same policy succeeds.

Abstract

Vision-language-action (VLA) models that master standard manipulation benchmarks still collapse when the robot starts from a slightly different pose or the camera is moved, and even training on perturbed data leaves it unsolved. We show that much of this failure does not lie in the policy at all: the robot's embodiment departs from the one the VLA was trained on, and the VLA, which never perceives its own body as such, cannot tell that anything has changed. We introduce Test-Time Embodiment Alignment *(TEA), which turns a VLA into a self-calibrating one. Like the forward models that let humans notice when their body behaves unexpectedly, TEA predicts how the training embodiment would realize each command and how the robot should appear to its camera, and corrects the VLA's commands and view only when the evidence shows that the body has departed from training. TEA is never told what has changed: it discovers the shift from the robot's own body during the task, leaves the policy untouched, works on top of whatever controller the robot uses, and needs no data from the deployment condition. Across five benchmarks, three robots and six VLAs, it lifts initial-state success by up to 59 points and camera-perturbed success by up to 21 points on LIBERO-Plus, recovers persistent action perturbations (38–60% to 81–88% under RobustVLA's action bias), and carries over to a dual-arm robot and to the Google Robot in SimplerEnv, while leaving unperturbed performance intact.*

1. Introduction

Vision-language-action (VLA) models [1, 2, 14, 15, 25, 32] now solve almost every task of standard simulated manipulation suites such as LIBERO [21]. Yet robustness benchmarks reveal that this competence is fragile in two specific ways [9, 36]: on LIBERO-Plus, the four VLAs we evaluate succeed in only 29–76% of episodes when the robot starts from a perturbed initial state and, if trained from a fixed camera, in 36–71% when the camera is moved, while language, lighting or layout changes barely affect them (Tab. 1). The same two factors dominate on a dual-arm robot [36] and in SimplerEnv's protocol [19].

The prevailing response adds data: domain randomization [29], synthesized starts and viewpoints [28], or fine-

tuning on the target condition. This must be repeated for every model and deployment, and it does not close the gap: OpenVLA-OFT+ [15], fine-tuned on LIBERO-Plus-style perturbed data, is the most camera-robust policy we evaluate, yet it succeeds in only 29% of initial-state-perturbed episodes.

We take a different view. Tracing where each perturbation enters the closed loop of policy, controller and sensor, we find that much of the failure lies outside the policy: a perturbed start is dragged further before the policy's first command and carried into every target that follows (Fig. 3), and a moved camera shows an unchanged scene from a viewpoint the policy has never seen (Fig. 1). The task, the scene and the policy are unchanged; what has changed is the robot's embodiment, and the VLA, which sees images and proprioception but has no reference for what they should be, cannot notice.

Humans correct their movements with an internal forward model: every motor command comes with a prediction of its sensory consequence, and a mismatch signals that the body has changed. VLAs have no such loop, and we give them one. Our key observation is that the robot's own body is a calibration reference that is always available: how it moves in response to a command and how it appears at its measured joint angles are predictable in every step of every task, so calibration can happen during the task itself, without calibration targets, dedicated motions or stopping the robot. With *Test-Time Embodiment Alignment* (TEA), a VLA becomes *self-calibrating*. **State Alignment** learns command realization from the training demonstrations, restores the start the VLA was trained on and keeps its commands realized as in training throughout the episode; **View Alignment** recognises a moved camera from the robot's own rendered body and presents the scene from the training viewpoint. Because the deployment condition is unknown, a self-calibrating VLA must also know when not to correct: each correction must explain the robot better than the hypothesis that nothing has changed, so one TEA runs on every episode, perturbed or not, whereas correcting whenever possible recovers less or breaks episodes that needed no correction (Sec. 5.3).

Three properties set TEA apart. It discovers the shift instead of being told it: prior test-time corrections transfer a policy to a known robot or camera [5, 12], whereas TEA is given neither the perturbation nor its size. It is not a reset: it stays in closed loop with the VLA, so it also undoes shifts that persist, raising four VLAs from 38–60% to 81–88% under RobustVLA's constant action bias [10], where restoring the start alone barely helps. And it never opens the robot's controller: unlike retuning or disturbance observers [8], it acts only through the commands the VLA sends, so the same principle runs unchanged on an operational-space and a joint-position controller, while the VLA's weights are never updated and no data is collected in the deployment condition.

Our contributions are:

- **Self-calibrating VLAs:** the first framework in which a VLA discovers and undoes unknown shifts of its own embodiment, using its body as the reference, without fine-tuning, deployment data or controller access.
- **Diagnosis:** we show where the two most damaging benchmark perturbations enter the closed loop, through mechanisms shared across policies.
- **Evidence:** gains of up to 59 points across five benchmarks, three robots and six VLAs, without harming unperturbed performance.

2. Related Work

VLA policies and their robustness. VLAs map images and instructions to robot actions as discrete tokens [3, 14] or continuous chunks predicted by regression [15, 32] or flow matching [1, 2, 25], often after cross-embodiment pretraining [23, 24]. Camera pose is a known factor in the generalization gap of visual imitation [26, 33], and LIBERO-Plus [9], RoboTwin 2.0-Plus [36] and SimplerEnv [19] identify camera viewpoint and robot initial state as the most damaging perturbations. These benchmarks establish *that* policies fail; we trace *where*, and remove it without touching the policy.

Training-time and runtime adaptation. The common remedy is more varied training data: domain randomization [29], synthesized viewpoints [28] or robots [6], robot masking [18], or fine-tuning on perturbed data, as in OpenVLA-OFT+ [15]. Test-time adaptation [31] and residual policies [13, 27] need target-domain data, gradients or interaction. TEA needs none and complements all of them: it improves OFT+ as much as any policy.

Test-time correction around a fixed policy. BYOVLA [11] edits task-irrelevant image regions, and AnyCamVLA [12] re-renders images from the training viewpoint, given the ground-truth poses of both cameras and a view synthesizer fine-tuned on the benchmark's scenes. Mirage [5] transfers a policy to a different, known robot by cross-painting and learned forward dynamics, and G-PSMR [34] restores a VLA's image and proprioceptive streams but leaves the initial-state failure largely unresolved. These methods correct a shift that is given, a known target robot, known cameras or a fixed input statistic; TEA must first discover whether, and by how much, its own embodiment has shifted, and is the first to let a VLA do so from its own body and decide by itself when to act. Disturbance observers [8] do estimate unknown disturbances, but from inside the controller they are designed into; TEA

leaves the controller untouched and acts only through the VLA's commands.

Calibration and registration. The camera-to-robot pose can be estimated from a single image with learned keypoints [16, 17] or rendering [4, 22], and system identification fits a robot's dynamics. TEA uses the body as evidence of whether a VLA's view and commands have departed from training, without per-robot training.

3. Method

Setting. A policy π maps an observation $o_t = (I_t, s_t, \ell)$ (camera image(s), proprioceptive state, instruction) to a chunk of W commands that a low-level controller executes. We are given the training demonstrations $\mathcal{D}$, the robot's kinematic and visual model, its joint readings and the camera intrinsics K, assets that any robot used to collect demonstrations has; from them, both modules predict the same reference, the robot's own body: its motion in response to commands and its appearance at the measured joint angles. TEA may modify only (i) the commands sent to the controller (State Alignment) and (ii) the image passed to π (View Alignment) (Fig. 2), and is never told which perturbation, if any, is present. No data from the deployment condition or the evaluation is used for fitting or tuning, no thresholds are fitted, and TEA never chooses a task target: its corrections are low-level additions to the policy's commands.

3.1. State Alignment

Forward model of command realization. From $\mathcal{D}$ we fit a small MLP f that predicts the realized end-effector motion y_t from a command, its three predecessors and the state s_t, capturing how the nominal system realizes commands. At runtime we evaluate f on the executed, corrected commands, $m_t = f(\tilde{a}_{t:t-3}, s_t)$; a shifted embodiment (*e.g.* commands realized differently than in training) shows up as a systematic residual $y_t - m_t$.

Relative evidence instead of thresholds. For translation and rotation separately, we keep leaky statistics with decay $\lambda = 1 - 1/W$ and compare predictors by their windowed one-step errors, each with its own least-squares scale and offset. Three comparisons decide what to do:

- *Trust*: m must explain the observed motion better than the model-free baseline "motion $\propto$ commanded offset"; otherwise nothing is corrected.
- *Gain*: if m rescaled by an online gain k predicts better than m itself, the command is divided by $k \in [0.25, 4]$.
- *Drift*: if adding the leaky sum e of unexplained motion $k\,m_t - y_t$ improves prediction, $e/(W\rho)$ is cancelled per step, where ρ is the realization gain of the demonstrations.

Each comparison pits two predictors fitted on the same window, so TEA defaults to doing nothing when the embodiment behaves nominally or when f is outside its competence.

Restoring the training start. Most evaluators execute a few idle steps after reset, during which a perturbed start drifts further (Sec. 5.4). A settling model fitted on $\mathcal{D}$ predicts the pose the nominal arm would settle to; at the policy's first command, the displacement δ from this pose is added back as a correction that decays as $\lambda^j \delta/(W\rho)$. At that moment no online evidence exists yet, so the restoration relies on the demonstrations as a prior, and the trust test switches it off once the observed motion contradicts the model; the gain and drift corrections act only on evidence gathered during the episode. TEA sees the controller only through the robot's motion, so it applies to any controller; on RoboTwin's Aloha the same principle runs per arm in joint space (Suppl. Secs. 9 and 11).

3.2. View Alignment

Registration by render-and-compare. When the camera moves, the scene and the robot are unchanged; only the projection differs. The camera is assumed fixed within an episode: on its first observation, feature matches [20, 30] to a training-view reference give camera-pose hypotheses by PnP. Each candidate T is scored by rendering the robot at its measured joint angles from T and comparing it with the image around the robot: the robot is the one object whose 3D shape and pose are known exactly in every frame. The training extrinsics T_0 are calibrated once in the same way from demonstration images.

Evidence gate. Alignment is enabled only if the best candidate $\hat{T}$ explains the robot better than T_0 under the same score, a relative decision that needs no threshold. When the camera is unchanged, T_0 wins or the registered pose stays within about a centimetre of it, so the policy sees essentially its raw image. The gate decides whether the camera has moved, not whether a given policy benefits from alignment; View Alignment is therefore meant for policies trained from a fixed camera, which is known from the policy's training recipe before deployment.

Geometric re-projection. Each pixel receives a metric depth (support plane, rendered robot depth, or monocular depth [35] anchored to both) and is re-projected from $\hat{T}$ to T_0: $\mathbf{u}_0 = K\,\pi\big(T_0 \hat{T}^{-1} D(\mathbf{u})\, K^{-1}\tilde{\mathbf{u}}\big)$. Robot parts visible only from T_0 are rendered at the current joint angles, removing the strongest parallax error, and remaining holes are filled

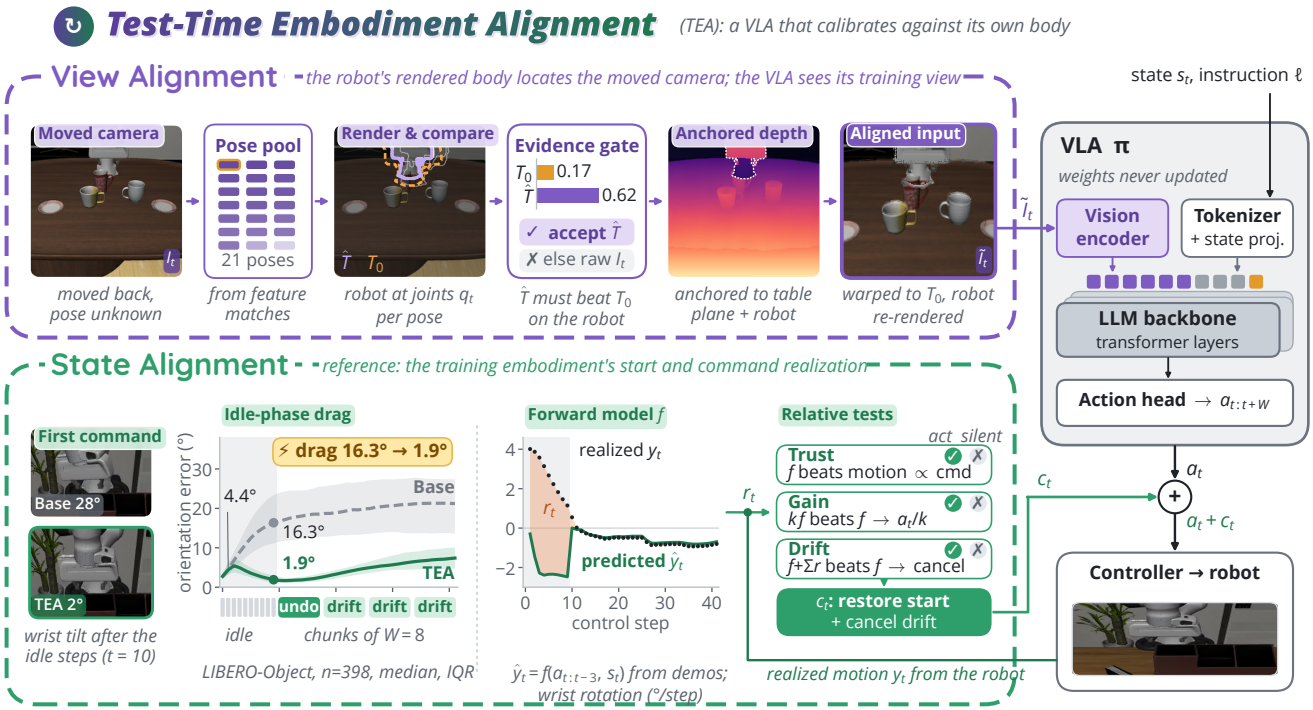

Figure 2. **Overview of Test-Time Embodiment Alignment (TEA).** The VLA is used as released (right). TEA aligns the image it sees (View Alignment, top) and the commands it sends (State Alignment, bottom), using the robot's own body as the reference: the rendered robot locates a moved camera, and a forward model of the training embodiment exposes and cancels unexplained motion. Every correction must beat the hypothesis that nothing has changed. Images and numbers are from logged LIBERO-Plus episodes.

from the current image. No generative model and no training image is involved.

4. Experiments

We evaluate one TEA configuration on five benchmarks, three simulators, three robots and six VLAs, with over 195,000 episodes (Suppl. Tab. 20). We ask whether TEA recovers the targeted perturbations across policies (Q1), does no harm on clean and untargeted conditions, which the base policies already handle, although it runs on every episode (Q2), owes each gain to the component designed for it (Q3), works for the reason we claim (Q4), and transfers to other robots and policies (Q5).

Benchmarks and policies. *LIBERO-Plus* [9] perturbs the four LIBERO suites along seven categories at five levels (10,030 episodes per policy); the original *LIBERO* [21] is the clean reference. We evaluate four VLAs used as released: OpenVLA-OFT and OFT+ [15], the latter trained on LIBERO-Plus-style perturbed data, $\pi_{0.5}$ [25] and VLA-Adapter-Pro [32]. *RobustVLA* [10] perturbs the actions while the policy acts (constant bias, per-episode uniform offset, Gaussian noise, external push); unlike a shifted start, these persist, so they test whether the correction keeps working. *RoboTwin 2.0-Plus* [7, 36] perturbs the initial joints of a dual-arm Aloha over 50 tasks, with $\pi_{0.5}$ (joint deltas added to the observed state) and Fast-WAM (absolute joint targets). *SimplerEnv* [19] provides the Google Robot with its official alternative cameras and 15 scene variants, with GR00T N1.6 [1]. Full TEA runs on LIBERO-Plus; RobustVLA and RoboTwin test State Alignment, whose camera is nominal, and SimplerEnv, whose real-robot demonstrations lack joint states, tests View Alignment.

Protocol. Every comparison is paired: the base policy and the policy with TEA run identical episodes (initial state, perturbation and seed), with common random numbers for stochastic policies, and we report exact McNemar tests on the discordant pairs. One TEA configuration is used for every policy within a benchmark; Suppl. Sec. 10 gives the full protocol.

5. Results

5.1. Recovery under the targeted perturbations (Q1)

One TEA configuration, never adapted to a particular policy, is applied to four VLAs that differ in backbone, ac-

	Robot initial state					Camera viewpoint					All 7 categories	
Policy	Spatial	Object	Goal	Long	Avg.	Spatial	Object	Goal	Long	Avg.	Untargeted	Total
OpenVLA-OFT	38.0	33.9	24.9	39.2	33.8	90.7	43.2	62.3	37.7	57.8	82.8	71.2
+ TEA	**90.6**	**90.5**	**83.6**	**82.7**	**86.7** (+52.9)	**93.9**	**49.5**	**66.7**	**56.8**	**66.2** (+8.4)	**83.3** (+0.5)	**81.1** (+9.8)
OpenVLA-OFT+	31.7	24.6	21.0	39.7	29.1	97.6	98.5	97.8	87.4	95.2	89.0	80.7
+ TEA†	**85.4**	**93.2**	**88.5**	**83.5**	**87.7** (+58.6)	**98.1**	**99.7**	95.8	85.2	94.6 (−0.6)	**89.8** (+0.8)	**90.2** (+9.5)
$\pi_{0.5}$	87.1	71.6	71.1	74.3	75.7	73.9	87.1	75.2	48.7	70.9	89.9	84.7
+ TEA	**96.3**	**96.2**	**95.4**	**87.0**	**93.7** (+18.0)	**87.2**	85.1	**81.1**	**62.1**	**78.5** (+7.6)	89.7 (−0.2)	**88.5** (+3.8)
VLA-Adapter-Pro	56.9	38.2	26.7	33.6	38.2	71.8	3.8	65.4	7.2	36.4	68.9	59.0
+ TEA	**96.0**	**93.7**	**74.8**	**74.0**	**84.3** (+46.1)	**91.5**	**32.3**	**72.8**	**36.0**	**57.5** (+21.1)	**69.2** (+0.3)	**69.7** (+10.7)

Table 1. **LIBERO-Plus success rates (%) in the two targeted perturbation categories, per suite.** Each policy is evaluated as released (first row) and wrapped by TEA (second row) on identical episodes. Untargeted: the other five categories pooled. Parentheses: change in points. Every category and suite: Suppl. Tab. 6. †OFT+ is trained with camera variation, so TEA applies State Alignment only (Sec. 3.2).

	Robot initial state, by level					Camera, by type		
Policy	L1	L2	L3	L4	L5	Rot.	Dist.	Orbit
OFT	90.7	65.8	23.3	5.3	1.4	70.1	59.1	53.7
+ TEA	**96.0**	**90.1**	**86.6**	**85.3**	**78.6**	**89.8**	**81.2**	**54.5**
OFT+	76.6	50.0	23.0	7.0	2.6	94.2	93.6	96.0
+ TEA†	**96.0**	**92.1**	**85.8**	**85.3**	**82.3**	94.2	**94.6**	94.7
$\pi_{0.5}$	97.2	90.8	84.0	63.9	48.7	93.5	74.8	63.0
+ TEA	**97.6**	**96.2**	**93.9**	**91.6**	**90.3**	**96.9**	**93.0**	**68.5**
Pro	85.1	61.0	36.6	15.1	6.6	37.8	29.4	38.2
+ TEA	**96.0**	**86.6**	**81.8**	**78.2**	**81.5**	**90.5**	**71.6**	**43.3**

Table 2. **Targeted categories by difficulty level and camera perturbation type** (LIBERO-Plus, success %, suites pooled). Pro: VLA-Adapter-Pro; †OFT+: State Alignment only. Camera by level: Suppl. Tab. 7.

tion head and action space (Tab. 1; every category in Suppl. Tab. 6). Although TEA is never told whether, or how, an episode is perturbed, it raises total success for every one of the 16 suite–policy combinations, and its gains fall precisely on the two categories it was designed for. Under robot-initial-state perturbation, policies that fail most episodes become reliable: from as low as 21.0% (OFT+, Goal) to 74–96% on every combination, with pooled gains of +46.1 points for VLA-Adapter-Pro, +52.9 for OFT, +58.6 for OFT+ and +18.0 for $\pi_{0.5}$ (all $p < 10^{-48}$). The largest gain belongs to OFT+, the one policy fine-tuned on perturbed data: what training on perturbed data could not remove, aligning the embodiment does. Under camera perturbation, View Alignment adds +21.1 (Pro), +8.4 (OFT) and +7.6 ($\pi_{0.5}$) points for the three policies trained from a fixed camera (all $p < 10^{-10}$), without ever being given the new camera pose.

The harder the perturbation, the larger the gain. Base success collapses with the initial-state level for every policy, to 1.4–48.7% at level 5 (Tab. 2); with TEA, the hardest level stays at 78.6–90.3%, nearly as solvable as the easiest. TEA does not soften the perturbation; it removes it. Under camera perturbation, rotations and distance changes, which the robot registers almost exactly from its own body, gain up to +52.7 and +42.2 points (Pro), and pooled over all seven categories the gain grows with the level for every policy (Suppl. Tab. 14). Once its inputs match training, the policy also acts more directly: initial-state episodes solved by both arms become 5–13 steps shorter.

5.2. Clean and untargeted conditions (Q2)

Unperturbed and untargeted episodes are not what TEA, or any robustness method, sets out to improve: the base policies already solve 96.5–97.3% of clean LIBERO episodes. There, a module that cannot know whether an episode is perturbed must simply do no harm, and TEA does. On the original LIBERO, no policy changes significantly (Pro: 97.2→97.5%; Suppl. Tab. 9), and on LIBERO-Plus no untargeted category drops by more than two points pooled over suites; over all 6,881 untargeted episodes, the paired 95% bootstrap interval bounds the aggregate cost at 0.8 points for every policy (Suppl. Tab. 11). This is not inertness: State Alignment stays active on 13–43% of untargeted steps without changing success. Without the evidence gate, re-projecting views whose camera never moved lowers Pro's 6,881 untargeted episodes from 69.2% to 67.3% ($p = 8 \times 10^{-8}$), below the base policy, whereas with it TEA matches the base policy.

5.3. Attribution to modules and components (Q3)

Each module owns its perturbation. For Pro, State Alignment alone reproduces the full initial-state gain without affecting camera success, View Alignment alone reproduces the camera gain without affecting initial-state success, and combining them neither helps nor hurts (Tab. 3a; 8 comparisons, all $p \geq 0.27$). The same split holds for $\pi_{0.5}$ despite its stochastic sampling, so the modules can be deployed separately, as on RoboTwin and SimplerEnv.

(a) Module attribution (pooled over episodes).

Policy	Method	Camera	Robot init.
Pro	Base policy	36.4	38.2
	+ State Alignment	37.0	**84.9**
	+ View Alignment	**58.1**	38.8
	+ TEA (both)	57.5	84.3
$\pi_{0.5}$	Base policy	70.9	75.7
	+ State Alignment	72.4	92.5
	+ View Alignment	77.1	76.3
	+ TEA (both)	**78.5**	**93.7**

(b) View Alignment components (camera).

Variant	Spatial	Object	Goal	Long	Avg.
VLA-Adapter-Pro					
TEA (full)	91.5	32.3	72.8	36.0	57.5
w/o evidence gate	91.2	32.3	72.5	36.3	57.5
fill from training view	91.8	31.3	74.5	38.2	58.3
single pose (no pool)	90.2	30.8	72.5	42.2	58.4
w/o robot re-rendering	76.6	15.2	73.0	20.3	45.7
Base policy	71.8	3.8	65.4	7.2	36.4
OpenVLA-OFT					
TEA (full)	93.9	49.5	66.7	56.8	66.2
w/o robot re-rendering	96.3	46.0	67.9	49.2	64.2
Base policy	90.7	43.2	62.3	37.7	57.8

(c) State Alignment components (VLA-Adapter-Pro, robot init.).

Variant	Spatial	Object	Goal	Long	Avg.
Relative tests only	96.0	78.1	70.7	68.2	77.7
+ warm-up forward model	96.9	89.7	64.3	68.2	79.2
+ start restoration (full)	97.1	93.5	75.3	75.3	84.9
demo-fitted thresholds	96.6	84.4	68.2	72.0	79.7
without evidence test	93.7	76.9	61.1	69.2	74.6
window W=4	95.7	89.4	70.9	78.6	83.2
window W=16	97.1	94.0	74.6	77.1	85.3
Base policy	56.9	38.2	26.7	33.6	38.2

(d) Simpler correction rules (VLA-Adapter-Pro).

Rule	Init.	G-Noise	L-Bg.	O-Lang.	Clean
Return to first pose	76.5	74.7	61.9	70.9	97
TEA	**84.9**	82.3	76.5	76.6	100
Base policy	38.2	85.0	75.4	74.6	100

Table 3. **Ablations on LIBERO-Plus**, success rates (%). Shaded: the full method; bold in (a): best per column within each policy. (a) Each module alone and combined, for two policies (per suite: Suppl. Tab. 8). (b) TEA with one View Alignment component changed. (c) State Alignment built up component by component (W=8), and with alternative choices. (d) Rules that act without relative evidence: undoing all motion back to the first observed pose, on initial-state (Init.) episodes and on untargeted and clean ones (G-Noise: Goal, sensor noise; L-Bg.: Long, background; O-Lang.: Object, language; Clean: LIBERO-Spatial, 100 episodes); red: significant decrease ($p < 0.05$). Homing the arm when its start pose leaves the demonstration distribution never triggers.

Knowing when not to correct is worth ten points. The relative tests alone already lift initial-state success from 38.2% to 77.7%, and the warm-up model and start restoration bring it to 84.9% (Tab. 3c). Given the same forward model and the same start restoration but correcting whenever a correction exists, success drops to 74.6%, on every suite ($p = 2 \times 10^{-20}$, 1,550 pairs). Thresholds fitted on demonstration statistics are worse and transfer poorly (16% of their gain from Spatial to Long), and the window is not tuned ($W = 16$ matches $W = 8$).

Simpler rules fail in instructive ways. Homing the arm when its start pose looks abnormal never triggers: the shift shows only in how the VLA's commands are realized, which only a model of the training embodiment can see (Tab. 3d). Undoing all motion back to the first observed pose recovers less than TEA (76.5% vs. 84.9%, $p = 5 \times 10^{-11}$) and breaks episodes that needed nothing (Goal sensor noise 85.0→74.7%, Long background 75.4→61.9%), which TEA leaves intact.

View Alignment needs the robot, not the true camera. Re-rendering the arm at its measured joints is what makes View Alignment work: generic depth-based re-projection, which treats the robot as static scene, loses 14.9–17.2 points on Spatial, Object and Long for Pro (all $p < 10^{-9}$), whereas the strictest safeguards, filling only from the current image and a single pose estimate, cost at most 2.2 points (Tab. 3b). Nor does View Alignment need to be told where the camera is: given the ground-truth pose that methods assuming a calibrated test camera require, synthesis-only camera success rises only from 58.1% to 61.3%, so the pose the robot infers from its own body already delivers 87% of the gain of a known camera.

5.4. Why the modules work (Q4)

State Alignment returns the policy to its training start… In step-level logs of Pro, OFT and $\pi_{0.5}$ (1,550 episodes each), the median wrist orientation error grows from 4.4–4.8° at the first idle step to 15.6–16.9° at the policy's first command, identically for all three policies: a property of the system, not of any policy. State Alignment brings it to 1.3–2.1°. Without correction, success falls steeply with the drag, from 59.8% to 16.8% for Pro and from 92.1% to

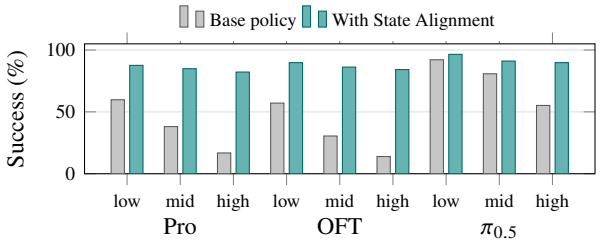

Figure 3. **Success by tercile of idle-phase drift** (position error at the policy's first command), robot-initial-state perturbation (Pro: VLA-Adapter-Pro, OFT: OpenVLA-OFT), four suites pooled (1,550 episodes per policy and arm; for OpenVLA-OFT, full TEA).

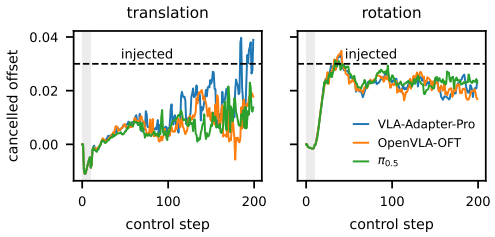

Figure 4. **State Alignment discovers an unknown action offset from the robot's own motion.** Offset cancelled by State Alignment ($a - x$, policy minus executed command, per arm dimension, averaged over dimensions and 80 episodes per policy) under RobustVLA's constant +0.03 bias; shaded: idle steps. The bias is never given to TEA.

55.2% even for $\pi_{0.5}$ between the lowest and highest tercile; with correction it is nearly flat (87.6 / 84.9 / 82.2% and 96.5 / 91.1 / 89.8%, Fig. 3). Once the policy starts where it was trained, it succeeds however far it had been pulled.

...and keeps correcting while the VLA acts. Under initial-state perturbation, the gain or drift correction is active on 72–75% of all control steps, almost all after the VLA has started acting: State Alignment is a closed loop around the VLA, not an initialization step.

View Alignment returns the training-camera behaviour. At the same simulator state, the aligned input moves Pro's action toward the action it takes from its training camera in 91% of queries for rotations, 83% for distance changes and 66% for orbits (4,416 queries), and OFT, with a different backbone and action head, behaves alike (89 / 78 / 62%).

5.5. Shifts that persist through the episode

A shift set at reset cannot tell a self-calibrating VLA from one that merely fixes its start; RobustVLA's perturbations, which act on every command while the VLA runs, can

Policy	Action bias	Action uniform	Action noise	External force
OpenVLA-OFT	44.2	58.9	22.4	28.6
+ start only	44.0 (−0.2)	60.9 (+2.0)	23.8 (+1.4)	30.2 (+1.6)
+ State	**86.5** (+42.2)	**93.8** (+34.9)	**30.2** (+7.9)	**31.9** (+3.2)
OpenVLA-OFT+	38.0	56.9	26.4	29.2
+ start only	42.5 (+4.5)	62.0 (+5.1)	25.5 (−0.9)	31.2 (+2.0)
+ State	**80.8** (+42.8)	**91.0** (+34.1)	**33.8** (+7.4)	**33.5** (+4.2)
$\pi_{0.5}$	60.0	71.0	46.0	45.2
+ start only	60.5 (+0.5)	72.2 (+1.2)	47.4 (+1.4)	48.8 (+3.5)
+ State	**87.8** (+27.8)	**92.6** (+21.6)	**52.5** (+6.5)	**51.2** (+6.0)
VLA-Adapter-Pro	38.8	56.6	23.9	27.8
+ start only	38.8 (+0.0)	59.0 (+2.4)	24.1 (+0.2)	29.5 (+1.8)
+ State	**83.8** (+45.0)	**89.4** (+32.8)	**30.5** (+6.6)	**33.0** (+5.2)

Table 4. **Continuous action and force perturbations** of RobustVLA [10] on LIBERO, success rates (%) over the four suites (400 paired episodes per cell; 800 over two perturbation seeds for the random perturbations, while the constant bias draws nothing and is identical across seeds). The perturbation acts after TEA, on the executed commands or the end effector, so the policy and TEA see only its effects. Start only: State Alignment's restoration of the training start alone, without its corrections during the episode; State: full State Alignment. Bold: paired McNemar $p < 0.05$; parentheses: change in points. Per suite and p-values: Suppl. Tab. 17.

(Tab. 4). Although every policy re-plans from a fresh observation at every query, a constant action bias drops the four policies to 38.0–60.0%; with State Alignment, which is never told the bias, all four recover to 80.8–87.8%. A per-episode uniform offset gives the same picture (56.6–71.0% to 89.4–93.8%), even for OFT+, trained on perturbed data. Restoring the start alone, by contrast, recovers nothing: under the bias the policies stay at 38.8–60.5%, and across all four perturbations it never adds more than 5.1 points, whereas State Alignment adds 21–45 points under the persistent offsets. TEA is not a reset. Nor are the gains an artifact of one benchmark's reset: RobustVLA is independent of the start, and on RoboTwin, whose controller tracks exactly, a start offset alone still derails the policy (Sec. 5.6).

Figure 4 shows how the offset is discovered. On the rotation dimensions, the cancelled offset reaches 80% of the injected bias 13–16 steps after the VLA starts acting and then holds, for all three policies logged, although nothing tells TEA its size or sign. The bias drives 40 of 160 task–policy pairs to 0/10; State Alignment brings back every one. Even zero-mean noise and a single push, which leave little to correct, improve every policy (+3.2 to +7.9 points, $p \leq 0.014$).

5.6. Other robots and policies (Q5)

To test whether the principles, not a LIBERO-specific implementation, carry the gains, we apply each module un-

(a) RoboTwin 2.0-Plus
Aloha-AgileX dual arm · State Alignment

Task family (tasks)	$\pi_{0.5}$ Base	$\pi_{0.5}$ +State	$\pi_{0.5}$ p	Fast-WAM Base	Fast-WAM +State	Fast-WAM p
Robot initial state, 10 episodes per task						
Place (17)	10.2	**20.5**	4.6×10^{-3}	56.5	**69.4**	4.6×10^{-3}
Pick & move (12)	7.8	**12.2**	0.23	57.1	**67.0**	0.08
Stack & rank (6)	10.2	**33.9**	1.2×10^{-4}	70.0	**85.0**	0.01
Articulated & tool (13)	22.8	**31.5**	0.04	71.2	68.0	0.54
Handover (2)	0.0	**5.0**	1.00	60.0	**70.0**	0.50
All tasks (50)	12.5	**22.4**	1.3×10^{-7}	62.2	**70.4**	4.6×10^{-4}
6 tasks, 50 ep. each	23.5	**39.1**	2.3×10^{-7}	82.7	**89.5**	1.2×10^{-3}
Clean, 10 episodes per task						
All tasks (50)	51.1	50.9	1.00	91.2	91.2	1.00

(b) SimplerEnv
Google Robot · GR00T N1.6 · View Alignment

Variant	Base	+View	p
Official camera variants, 3 repetitions			
Cam. 1, pick can	54.7	50.2	0.36
Cam. 1, move near	16.7	**27.8**	2.2×10^{-3}
Cam. 2, pick can	32.0	**46.7**	8.1×10^{-4}
Cam. 2, move near	38.3	**40.6**	0.68
All camera	36.3	**42.1**	6.3×10^{-3}
Official scene variants			
Background (4)	59.6	63.0	0.44
Lighting (4)	47.4	47.4	1.00
Distractors (3)	48.6	54.8	0.14
Table texture (4)	47.4	46.3	0.83
All (15)	50.9	52.7	0.34

Table 5. **Results on other robots and policies**, success rates (%) of the base policy and with the module (shaded), on identical episodes; bold: improvement. (a) RoboTwin 2.0-Plus by task family; $\pi_{0.5}$ predicts joint deltas and Fast-WAM absolute joint targets; 487 pairs per policy under perturbation; on clean episodes 482 of 483 ($\pi_{0.5}$) and 478 of 478 (Fast-WAM) pairs are identical (per task: Suppl. Tab. 16). (b) SimplerEnv variant-aggregation scenes; Cam. 1/2: AltGoogleCamera/AltGoogleCamera2; 810 camera and 1020 scene-variant pairs; in parentheses, the number of variants (per variant: Suppl. Tab. 15).

changed to other robots, simulators, controllers and policies (Tab. 5).

State Alignment on a dual-arm robot. RoboTwin's Aloha runs a joint-position controller, used without any access to its internals, that tracks exactly; yet the policy still fails, because all 27,500 training demonstrations start at the same home configuration and $\pi_{0.5}$ adds its predicted deltas to the observed state, so a start offset enters its first target. Since the home pose is known exactly, restoring it within the first chunk, without the trust test, nearly doubles $\pi_{0.5}$'s success over all 50 tasks (12.5→22.4%, $p = 1.3 \times 10^{-7}$) and improves every task family. Fast-WAM, which predicts absolute targets and itself removes part of the offset, still gains 8.2 points ($p = 4.6 \times 10^{-4}$). On clean episodes the module is inert: every Fast-WAM pair and all but one $\pi_{0.5}$ pair are identical (example in Suppl. Fig. 11).

View Alignment on the Google Robot. On SimplerEnv's official head-camera variants, the robot proposes camera candidates from its head-joint kinematics and verifies them by render-and-compare. Without any evaluation image, View Alignment raises GR00T N1.6 from 36.3% to 42.1% ($p = 0.006$), in each of three repetitions and exactly where the base policy collapses, while the 15 scene variants are unaffected ($p = 0.34$).

5.7. Conditions of use and cost

TEA needs only assets that any robot that collected demonstrations has (Suppl. Tab. 19); State Alignment assumes demonstrations recorded with the deployed controller. View Alignment's gate decides whether the camera has moved, not whether a particular policy benefits from the aligned view; it therefore serves policies trained from a fixed camera, the common case, and OFT+, trained with camera variation, runs State Alignment only. All experiments are in simulation, but corrupting what TEA reads with realistic measurement errors, sub-millimetre pose noise, 1° joint offsets or a 5% focal-length error, leaves its gains unchanged (Suppl. Sec. 8.2). TEA adds little cost: State Alignment 0.4 ms per control step, View Alignment 16 ms per policy query (beside 43 ms for the policy) and registration 0.3 s once at reset; an average control step takes 32.6 ms instead of 25.4 ms (Suppl. Tab. 12).

6. Conclusion

VLAs fail under a perturbed start or a moved camera not because they cannot do the task, but because their embodiment has departed from the one they were trained on, and they have no reference against which to notice it. TEA gives them that reference: their own body. Never told what has changed, a self-calibrating VLA discovers when its embodiment departs from training and corrects its commands and view only where the evidence demands it, without touching the policy or opening the controller. It restores policies that perturbed training could not fix, recovers persistent shifts where resetting cannot, and carries over to other robots and policies; self-calibration against one's own body is what embodied foundation models lack.

References

[1] Johan Bjorck et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025. We use the GR00T N1.6 fractal checkpoint. 1, 2, 4

[2] Kevin Black, Noah Brown, Danny Driess, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024. 1, 2

[3] Anthony Brohan et al. RT-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022. 2

[4] Linghao Chen, Yuzhe Qin, Xiaowei Zhou, and Hao Su. EasyHeC: Accurate and automatic hand-eye calibration via differentiable rendering and space exploration. *IEEE Robotics and Automation Letters*, 8(11):7234–7241, 2023. 3

[5] Lawrence Yunliang Chen, Kush Hari, Karthik Dharmarajan, Chenfeng Xu, Quan Vuong, and Ken Goldberg. Mirage: Cross-embodiment zero-shot policy transfer with cross-painting. In *Robotics: Science and Systems XX*, 2024. 2

[6] Lawrence Yunliang Chen, Chenfeng Xu, Karthik Dharmarajan, Richard Cheng, Kurt Keutzer, Masayoshi Tomizuka, Quan Vuong, and Ken Goldberg. RoVi-Aug: Robot and viewpoint augmentation for cross-embodiment robot learning. In *Proceedings of The 8th Conference on Robot Learning*, pages 209–233. PMLR, 2025. 2

[7] Tianxing Chen, Zanxin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, Weiliang Deng, Yubin Guo, Tian Nian, Xuanbing Xie, Qiangyu Chen, Kailun Su, Tianling Xu, Guodong Liu, Mengkang Hu, Huan-ang Gao, Kaixuan Wang, Zhixuan Liang, Yusen Qin, Xiaokang Yang, Ping Luo, and Yao Mu. RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. In *Proceedings of the 43rd International Conference on Machine Learning*, pages 13673–13699, 2026. 4, 9

[8] Wen-Hua Chen, Jun Yang, Lei Guo, and Shihua Li. Disturbance-observer-based control and related methods—an overview. *IEEE Transactions on Industrial Electronics*, 63(2):1083–1095, 2016. 2

[9] Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinzhe He, Shiduo Zhang, Zhaoye Fei, Jinlan Fu, Jingjing Gong, and Xipeng Qiu. LIBERO-Plus: A progressive robustness benchmark for visual-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026. 1, 2, 4, 9

[10] Jianing Guo, Zhenhong Wu, Chang Tu, Yiyao Ma, Xiangqi Kong, Zhiqian Liu, Jiaming Ji, Shuning Zhang, Yuanpei Chen, Kai Chen, Qi Dou, Yaodong Yang, Xianglong Liu, Huijie Zhao, Weifeng Lv, and Simin Li. RobustVLA: On robustness of vision-language-action model against multi-modal perturbations. In *International Conference on Learning Representations (ICLR)*, 2026. 2, 4, 7, 9

[11] Asher J. Hancock, Allen Z. Ren, and Anirudha Majumdar. Run-time observation interventions make vision-language-action models more visually robust. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 9499–9506, 2025. 2

[12] Hyeongjun Heo, Seungyeon Woo, Sang Min Kim, Junho Kim, Junho Lee, Yonghyeon Lee, and Young Min Kim. AnyCamVLA: Zero-shot camera adaptation for viewpoint robust vision-language-action models. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2026. 2

[13] Tobias Johannink, Shikhar Bahl, Ashvin Nair, Jianlan Luo, Avinash Kumar, Matthias Loskyll, Juan Aparicio Ojea, Eugen Solowjow, and Sergey Levine. Residual reinforcement learning for robot control. In *ICRA*, 2019. 2

[14] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024. 1, 2

[15] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025. 1, 2, 4

[16] Yann Labbé, Justin Carpentier, Mathieu Aubry, and Josef Sivic. Single-view robot pose and joint angle estimation via render & compare. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1654–1663, 2021. 3

[17] Timothy E. Lee, Jonathan Tremblay, Thang To, Jia Cheng, Terry Mosier, Oliver Kroemer, Dieter Fox, and Stan Birchfield. Camera-to-robot pose estimation from a single image. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 9426–9432, 2020. 3

[18] Marion Lepert, Ria Doshi, and Jeannette Bohg. SHADOW: Leveraging segmentation masks for cross-embodiment policy transfer. In *Proceedings of The 8th Conference on Robot Learning*, pages 3536–3550. PMLR, 2025. 2

[19] Xuanlin Li, Kyle Hsu, Jiayuan Gu, et al. Evaluating real-world robot manipulation policies in simulation. In *Conference on Robot Learning (CoRL)*, 2024. 1, 2, 4, 9

[20] Philipp Lindenberger, Paul-Edouard Sarlin, and Marc Pollefeys. LightGlue: Local feature matching at light speed. In *ICCV*, 2023. 3

[21] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *NeurIPS Datasets and Benchmarks Track*, 2023. 1, 4, 9

[22] Jingpei Lu, Florian Richter, and Michael C. Yip. Markerless camera-to-robot pose estimation via self-supervised sim-to-real transfer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 21296–21306, 2023. 3

[23] Octo Model Team. Octo: An open-source generalist robot policy. In *Robotics: Science and Systems (RSS)*, 2024. 2

[24] Open X-Embodiment Collaboration. Open X-Embodiment: Robotic learning datasets and RT-X models. *arXiv preprint arXiv:2310.08864*, 2023. 2

[25] Physical Intelligence, Kevin Black, Noah Brown, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. 1, 2, 4

[26] Wilbert Pumacay, Ishika Singh, Jiafei Duan, Ranjay Krishna, Jesse Thomason, and Dieter Fox. THE COLOSSEUM: A benchmark for evaluating generalization for robotic manipulation. In *Robotics: Science and Systems XX*, 2024. 2

[27] Tom Silver, Kelsey Allen, Josh Tenenbaum, and Leslie Kaelbling. Residual policy learning. *arXiv preprint arXiv:1812.06298*, 2018. 2

[28] Stephen Tian, Blake Wulfe, Kyle Sargent, Katherine Liu, Sergey Zakharov, Vitor Campagnolo Guizilini, and Jiajun Wu. View-invariant policy learning via zero-shot novel view synthesis. In *Proceedings of The 8th Conference on Robot Learning*, pages 1173–1193. PMLR, 2025. 1, 2

[29] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *IROS*, 2017. 1, 2

[30] Michał Tyszkiewicz, Pascal Fua, and Eduard Trulls. DISK: Learning local features with policy gradient. In *NeurIPS*, 2020. 3

[31] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. In *ICLR*, 2021. 2

[32] Yihao Wang, Pengxiang Ding, Lingxiao Li, Can Cui, Zirui Ge, Xinyang Tong, Wenxuan Song, Han Zhao, Wei Zhao, Pengxu Hou, Siteng Huang, Yifan Tang, Wenhui Wang, Ru Zhang, Jianyi Liu, and Donglin Wang. VLA-Adapter: An effective paradigm for tiny-scale vision-language-action model. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 18638–18646, 2026. 1, 2, 4

[33] Annie Xie, Lisa Lee, Ted Xiao, and Chelsea Finn. Decomposing the generalization gap in imitation learning for visual robotic manipulation. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 3153–3160, 2024. 2

[34] Zhiyin Yan and Gang Shen. Restoring the right stream: Training-free OOD robustness for vision–language–action policies. *Entropy*, 28(9):975, 2026. 2

[35] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything v2. In *NeurIPS*, 2024. 3

[36] Zhanguang Zhang, Zhiyuan Li, Behnam Rahmati, Rui Heng Yang, Yintao Ma, Amir Rasouli, Sajjad Pakdamansavoji, Yangzheng Wu, Lingfeng Zhang, Tongtong Cao, Feng Wen, Xinyu Wang, Xingyue Quan, and Yingxue Zhang. Do world action models generalize better than VLAs? A robustness study. *arXiv preprint arXiv:2603.22078*, 2026. 1, 2, 4, 9

Self-Calibrating VLAs: Test-Time Embodiment Alignment for Vision-Language-Action Models

Supplementary Material

7. Additional results

Registration accuracy. On LIBERO-Plus camera episodes (VLA-Adapter-Pro, four suites), the median camera-centre error is 1.4 cm for rotations (rotation-magnitude error 0.1°), 3.4 cm for distance changes (1.1°) and 16.6 cm for orbits (5.5°).

RobustVLA protocol. The RobustVLA perturbation wraps the simulator step, so it acts on the command after TEA, and the policy and TEA observe only its effects; the ten idle steps at reset are not perturbed. All perturbation randomness is drawn from generators seeded by the perturbation seed, task and episode, independently of the actions, so the base and corrected runs of an episode receive the same offsets, noise and pushes, which we verified by a digest of the realized perturbation stream. For the external force we follow the released code (one push per episode of 100–300 N in a random direction for 3–6 steps); the variant described in the paper text (200 N along $+x$, repeated every 40–50 steps) leaves every arm below 7% on VLA-Adapter-Pro and does not separate methods. The random perturbations (uniform, noise, push) are evaluated with two perturbation seeds and pooled; the constant bias draws nothing, and its runs with a second seed reproduce every episode exactly (success and length), as expected for the deterministic VLA-Adapter-Pro, so it is reported once. With the second seed, the base success under noise and push changes by 0.2–1.2 points for the deterministic OFT and VLA-Adapter-Pro and by 3.6 points for $\pi_{0.5}$, which samples its actions; under the uniform offset, whose draw changes with the seed, it changes by up to 7.3 points. The gain of State Alignment has the same sign with both seeds in every cell. Per-suite results are in Tab. 17.

8. Further evidence

This section collects, by kind, the evidence behind the main claims: robustness across perturbation strength, why the self-calibrating correction can run on every episode, robustness to TEA's own measurement errors, and qualitative examples.

8.1. Recovery grows with the perturbation

Figure 5 plots Tab. 7 by level. The base policies lose most where the perturbation is strongest, and this is exactly where TEA recovers most: at the hardest robot-initial-state level, OFT rises from 1.4% to 78.6%, OFT+ from 2.6% to 82.3% and VLA-Adapter-Pro from 6.6% to 81.5%. The curves with TEA are nearly flat, *i.e*. the self-calibrating VLAs behave almost as if the start had not been perturbed.

8.2. Robustness to TEA's own measurement errors

All experiments in the paper use exact joint readings and robot models. Table 18 corrupts what TEA reads. Errors at the level of real systems, sub-millimetre pose noise, joint offsets of 1° and a 5% error in the assumed focal length, leave success unchanged; even noise several times larger leaves State Alignment's recovery intact, because its evidence tests withhold corrections that the noisy motion does not support. A 3° offset on every joint, far beyond encoder calibration error, still keeps 82% of View Alignment's gain.

8.3. Qualitative examples

Figures 6, 7 and 9 show episodes in which the base policy fails and the same policy with TEA succeeds, for every suite, every type of camera change and three different VLAs. The aligned views are close to the training camera without the policy or TEA ever seeing that camera.

8.4. Fixing the start is not enough

Figure 8 contrasts restoring the start with self-calibration under the four RobustVLA perturbations. A benchmark whose shift is confined to the start cannot tell the two apart; a persistent shift can, and only the self-calibrating VLA recovers it.

8.5. Why the self-calibrating correction can run on every episode

TEA runs on every episode without knowing whether it is perturbed, so its decisions must leave unperturbed episodes alone. Figure 10 (left) shows that the evidence gate is what achieves this for View Alignment, and the right panel shows that the robot's own body is a sufficient source of the camera pose: a method that is told the camera would gain little.

9. Implementation details

Forward model. The MLP takes the scaled 7-D command, three lagged commands, the end-effector rotation (matrix), position, gripper state and joint angles. It is trained with an episode-level held-out split of the training demonstrations. On LIBERO, a warm-up model is fitted on demonstration transitions plus the evaluator's idle phase, rebuilt from the demonstrations' own first states, and a settling model predicts the remaining settling motion from the

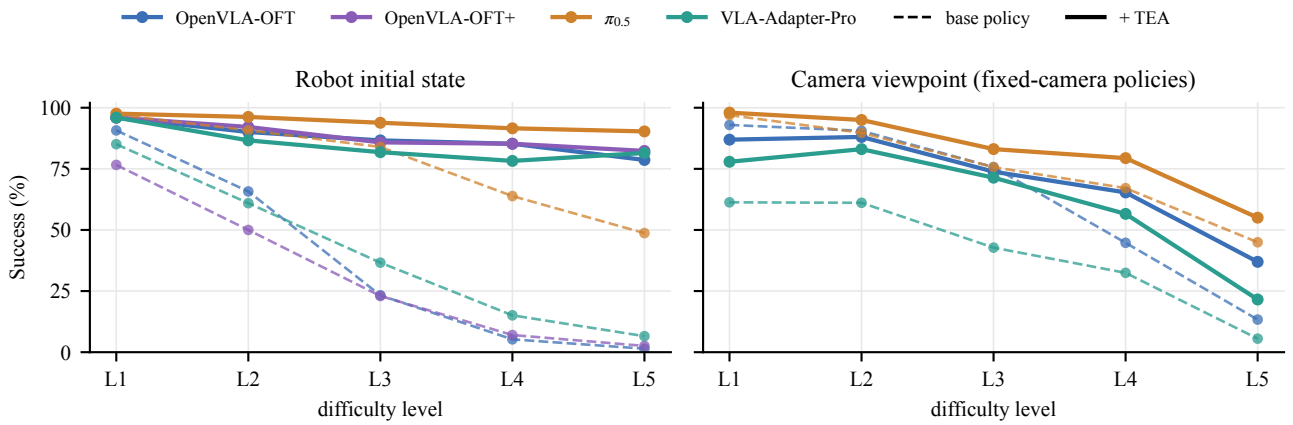

Figure 5. **Success by difficulty level** on LIBERO-Plus, suites pooled, base policy (dashed) and with TEA (solid), identical episodes. Under the robot-initial-state perturbation the base policies collapse with the level, to 1–49% at L5, while with TEA all four stay at 79–90%. Camera: the three fixed-camera policies (OFT+ is trained with camera variation, Sec. 5.7).

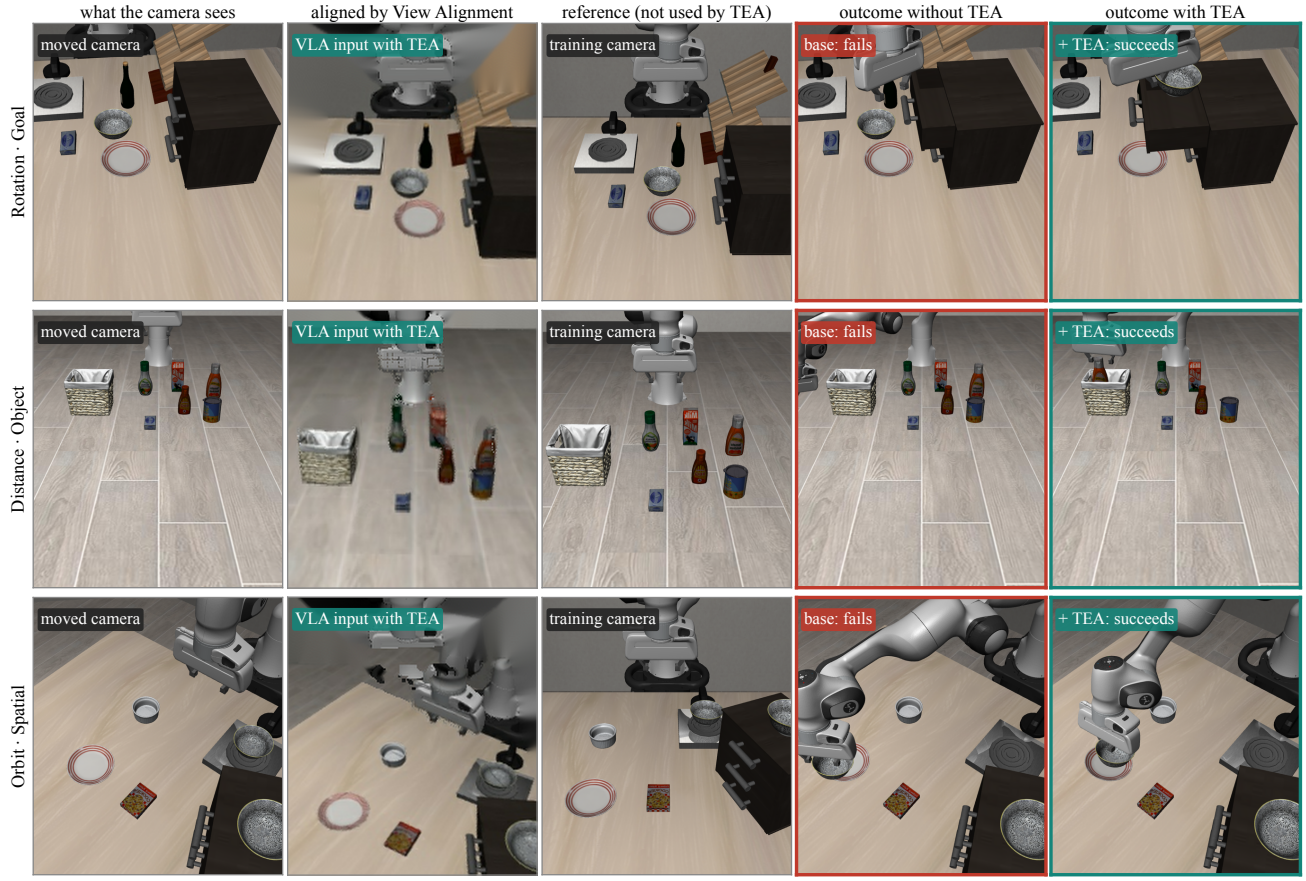

Figure 6. **Camera rotation, distance and orbit** (VLA-Adapter-Pro, hardest levels). From left: the moved view, the view TEA hands to the VLA, the training camera (rendered for reference; TEA never sees it), and the outcome of the base policy and of the policy with TEA on the identical episode.

Suite	Model	Camera	Robot	Language	Light	Background	Noise	Layout	Total
Spatial	OpenVLA-OFT	90.7→93.9 (+3.2)	38.0→90.6 (+52.6)	84.1→87.4 (+3.3)	99.7→99.7 (+0.0)	100.0→99.6 (−0.4)	94.9→93.4 (−1.4)	94.5→96.6 (+2.1)	85.3→94.0 (+8.8)
	OpenVLA-OFT+	97.6→98.1 (+0.5)	31.7→85.4 (+53.7)	93.8→95.9 (+2.1)	99.7→99.3 (−0.3)	98.4→99.2 (+0.8)	87.2→94.3 (+7.1)	95.8→96.1 (+0.3)	85.9→95.3 (+9.4)
	$\pi_{0.5}$	73.9→87.2 (+13.3)	87.1→96.3 (+9.1)	96.4→94.6 (−1.8)	99.0→98.6 (−0.3)	100.0→99.6 (−0.4)	93.7→92.9 (−0.9)	99.2→97.7 (−1.6)	92.3→95.0 (+2.7)
	VLA-Adapter-Pro	71.8→91.5 (+19.7)	56.9→96.0 (+39.1)	91.5→92.6 (+1.0)	100.0→99.7 (−0.3)	99.2→99.2 (+0.0)	88.6→86.9 (−1.7)	94.3→93.8 (−0.5)	85.3→93.8 (+8.6)
Object	OpenVLA-OFT	43.2→49.5 (+6.3)	33.9→90.5 (+56.5)	98.3→100.0 (+1.7)	89.6→88.9 (−0.7)	96.8→96.8 (+0.0)	67.3→68.2 (+0.9)	71.5→72.0 (+0.5)	68.8→79.1 (+10.3)
	OpenVLA-OFT+	98.5→99.7 (+1.3)	24.6→93.2 (+68.6)	100.0→99.7 (−0.3)	99.3→100.0 (+0.7)	99.6→98.8 (−0.8)	99.8→99.3 (−0.5)	78.2→78.7 (+0.5)	84.2→95.2 (+11.0)
	$\pi_{0.5}$	87.1→85.1 (−2.0)	71.6→96.2 (+24.6)	91.8→92.9 (+1.1)	99.0→99.0 (+0.0)	99.6→100.0 (+0.4)	94.1→96.0 (+1.9)	90.3→90.1 (−0.2)	89.6→93.7 (+4.1)
	VLA-Adapter-Pro	3.8→32.3 (+28.5)	38.2→93.7 (+55.5)	74.6→76.6 (+2.0)	54.9→57.6 (+2.7)	82.3→81.5 (−0.8)	29.4→28.7 (−0.7)	64.3→65.8 (+1.5)	46.9→60.8 (+13.9)
Goal	OpenVLA-OFT	62.3→66.7 (+4.4)	24.9→83.6 (+58.7)	60.2→55.1 (−5.1)	91.0→98.6 (+7.5)	96.8→95.4 (−1.4)	71.8→71.2 (−0.5)	64.7→64.0 (−0.7)	64.7→74.3 (+9.6)
	OpenVLA-OFT+	97.8→95.8 (−2.0)	21.0→88.5 (+67.5)	57.8→57.1 (−0.7)	88.5→97.1 (+8.6)	99.3→94.3 (−5.0)	97.9→96.3 (−1.6)	56.0→55.5 (−0.5)	71.7→82.0 (+10.3)
	$\pi_{0.5}$	75.2→81.1 (+5.9)	71.1→95.4 (+24.2)	71.5→71.7 (+0.2)	96.4→98.2 (+1.8)	93.6→96.4 (+2.8)	87.6→87.3 (−0.3)	70.6→72.7 (+2.1)	79.3→84.9 (+5.6)
	VLA-Adapter-Pro	65.4→72.8 (+7.4)	26.7→74.8 (+48.2)	45.4→42.7 (−2.7)	68.5→73.5 (+5.0)	49.1→52.7 (+3.6)	85.0→82.3 (−2.6)	51.1→51.1 (+0.0)	55.2→64.1 (+8.9)
Long	OpenVLA-OFT	37.7→56.8 (+19.1)	39.2→82.7 (+43.5)	92.4→85.6 (−6.8)	90.5→93.4 (+2.9)	88.2→88.9 (+0.7)	60.1→67.0 (+6.9)	80.1→80.4 (+0.3)	67.1→77.6 (+10.6)
	OpenVLA-OFT+	87.4→85.2 (−2.1)	39.7→83.5 (+43.8)	94.0→95.8 (+1.8)	89.1→95.6 (+6.6)	94.5→95.2 (+0.7)	93.8→91.8 (−2.0)	75.0→76.9 (+1.9)	81.5→89.0 (+7.4)
	$\pi_{0.5}$	48.7→62.1 (+13.4)	74.3→87.0 (+12.7)	86.2→86.9 (+0.8)	96.0→90.9 (−5.1)	92.0→91.0 (−1.0)	75.1→71.0 (−4.0)	87.2→87.2 (+0.0)	78.0→80.9 (+2.9)
	VLA-Adapter-Pro	7.2→36.0 (+28.9)	33.6→74.0 (+40.5)	84.9→84.3 (−0.5)	59.5→59.9 (+0.4)	75.4→76.5 (+1.0)	34.5→36.1 (+1.6)	75.6→74.4 (−1.3)	50.0→61.3 (+11.3)

Table 6. LIBERO-Plus success rates (%) for every suite, policy and perturbation category, base policy → with TEA. Parentheses: change in points.

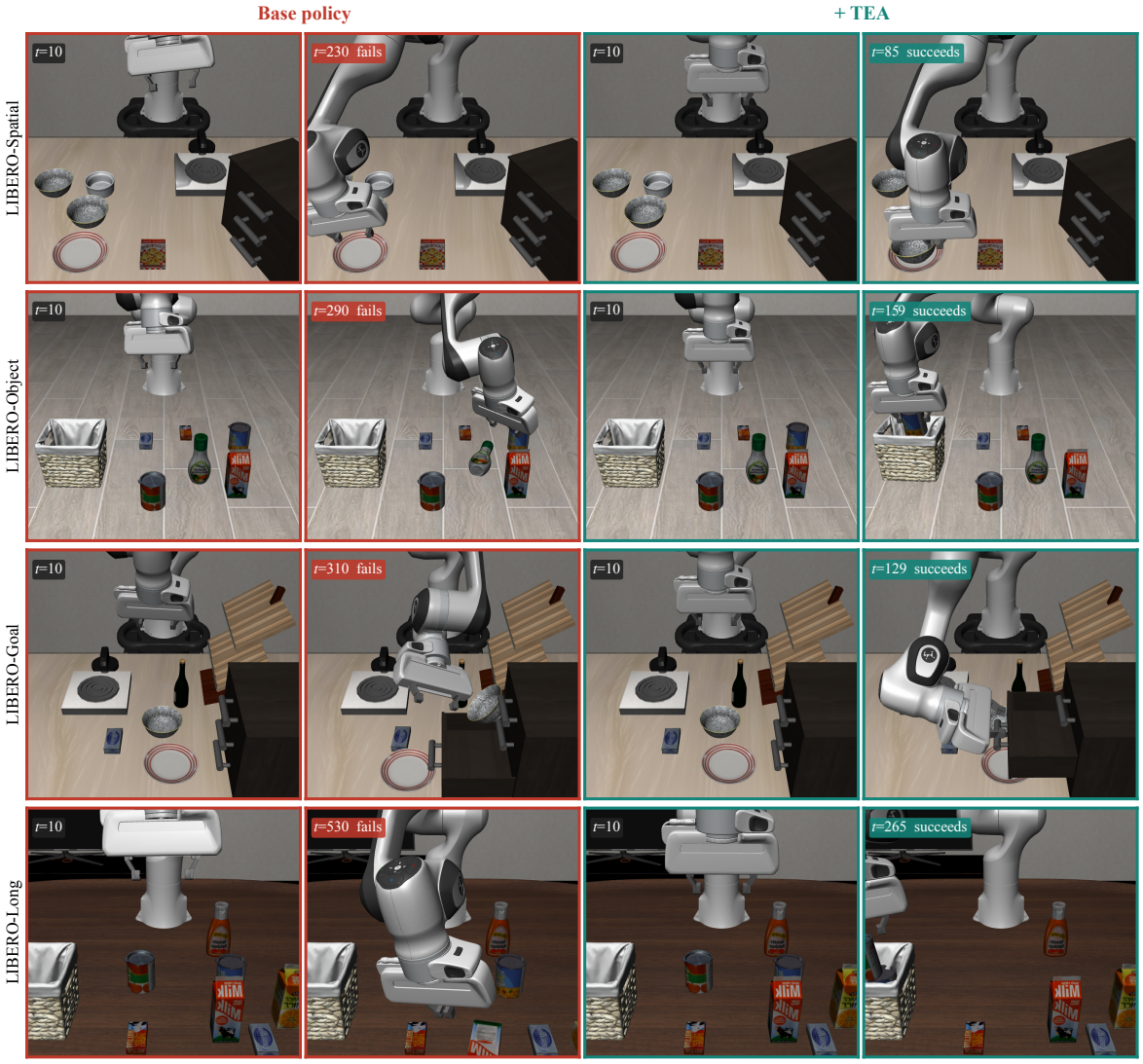

Figure 7. **Robot initial state, hardest level (L5), one task per suite** (VLA-Adapter-Pro, identical episodes, frames re-rendered from the recorded simulator states). The base policy never recovers from the shifted start; with TEA, the same policy completes every task.

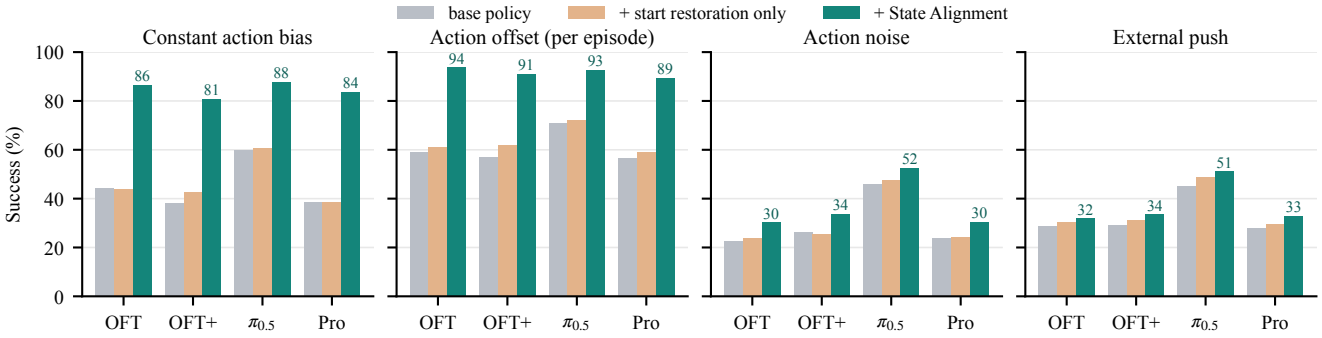

Figure 8. **RobustVLA perturbations**, four policies, suites pooled (data of Tab. 4). Restoring the training start alone leaves every base policy where it was; only the full State Alignment, which keeps correcting throughout the episode, recovers the persistent offsets.

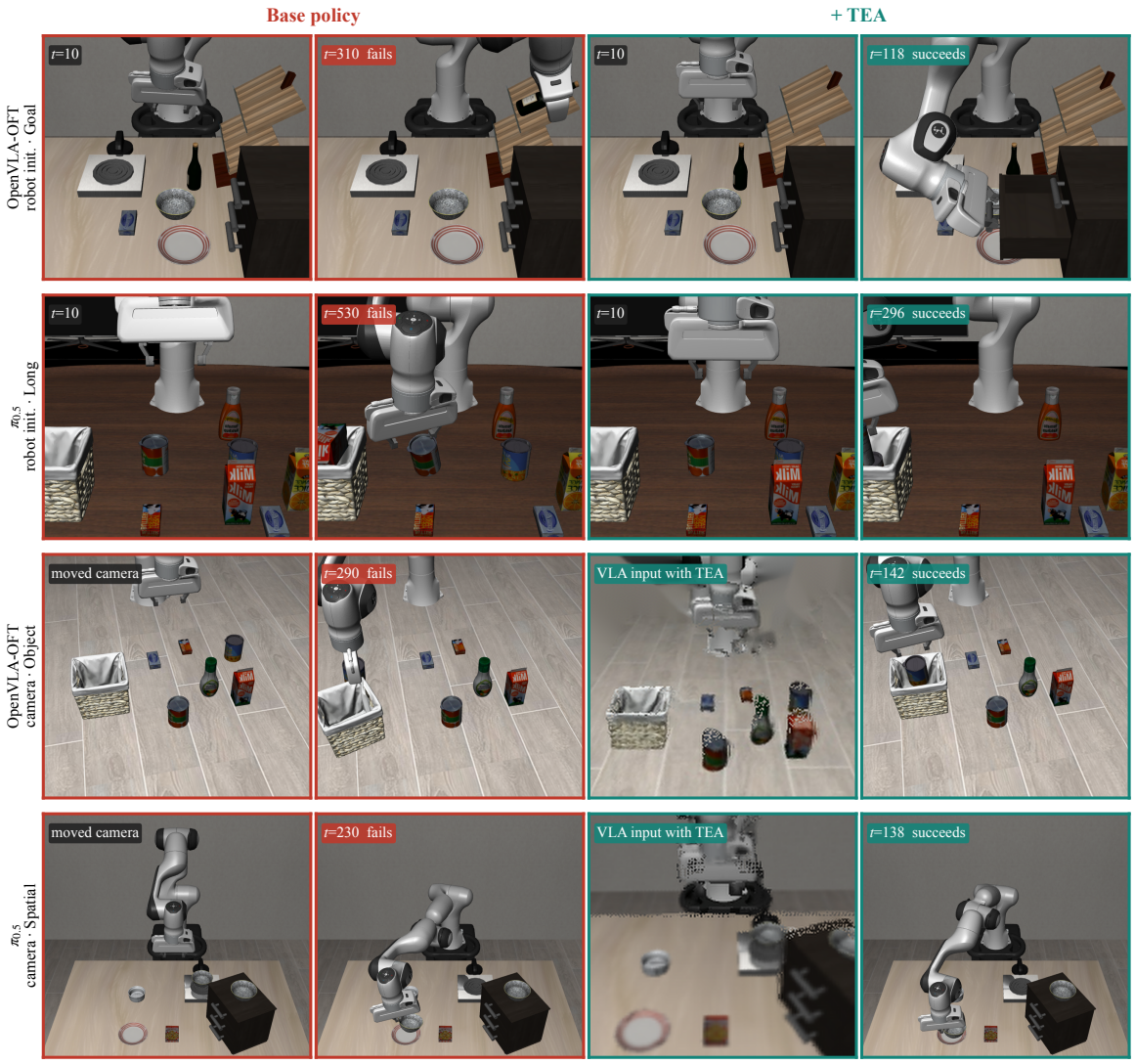

Figure 9. **The same correction for other VLAs** (hardest levels, identical episodes). OpenVLA-OFT and $\pi_{0.5}$ fail on the same robot-initial-state tasks as VLA-Adapter-Pro (Goal and Long rows of Fig. 7) and on moved cameras; with one TEA configuration, all succeed.

Policy	Robot initial state, by level					Camera viewpoint, by level					Camera viewpoint, by type		
	L1	L2	L3	L4	L5	L1	L2	L3	L4	L5	Rotation	Distance	Orbit
OpenVLA-OFT	90.7	65.8	23.3	5.3	1.4	93.0	90.6	75.7	44.7	13.3	70.1	59.1	53.7
+ TEA	**96.0**	**90.1**	**86.6**	**85.3**	**78.6**	86.9	88.1	73.8	**65.4**	**37.0**	**89.8**	**81.2**	**54.5**
OpenVLA-OFT+	76.6	50.0	23.0	7.0	2.6	99.0	98.6	96.9	93.9	90.6	94.2	93.6	96.0
+ TEA†	**96.0**	**92.1**	**85.8**	**85.3**	**82.3**	98.5	98.6	**97.2**	**94.3**	88.3	94.2	**94.6**	94.7
$\pi_{0.5}$	97.2	90.8	84.0	63.9	48.7	97.0	89.7	75.7	67.1	45.0	93.5	74.8	63.0
+ TEA	**97.6**	**96.2**	**93.9**	**91.6**	**90.3**	**98.0**	**95.0**	**83.1**	**79.4**	**55.0**	**96.9**	**93.0**	**68.5**
VLA-Adapter-Pro	85.1	61.0	36.6	15.1	6.6	61.3	61.1	42.8	32.5	5.5	37.8	29.4	38.2
+ TEA	**96.0**	**86.6**	**81.8**	**78.2**	**81.5**	**77.9**	**83.1**	**71.4**	**56.6**	**21.6**	**90.5**	**71.6**	**43.3**

Table 7. **LIBERO-Plus targeted categories by difficulty level and camera perturbation type**, success rates (%), suites pooled. Episodes per policy: robot initial state 1550, camera 1599 (rotation 294, distance 313, orbit 992). †OFT+: State Alignment only.

Policy	Method	Camera viewpoint					Robot initial state				
		Spatial	Object	Goal	Long	Avg.	Spatial	Object	Goal	Long	Avg.
VLA-Adapter-Pro	Base policy	71.8	3.8	65.4	7.2	36.4	56.9	38.2	26.7	33.6	38.2
	+ State Alignment	72.3	4.3	66.9	6.9	37.0	**97.1**	93.5	**75.3**	**75.3**	**84.9**
	+ View Alignment	90.7	**32.3**	**74.8**	**37.0**	**58.1**	57.7	38.7	27.6	33.6	38.8
	+ TEA (State + View)	**91.5**	**32.3**	72.8	36.0	57.5	96.0	**93.7**	74.8	74.0	84.3
$\pi_{0.5}$	Base policy	73.9	87.1	75.2	48.7	70.9	87.1	71.6	71.1	74.3	75.7
	+ State Alignment	74.5	**92.4**	75.2	48.7	72.4	**96.9**	95.0	93.2	85.2	92.5
	+ View Alignment	86.2	81.6	78.9	**63.0**	77.1	87.1	73.6	72.4	73.3	76.3
	+ TEA (State + View)	**87.2**	85.1	**81.1**	62.1	**78.5**	96.3	**96.2**	**95.4**	**87.0**	**93.7**

Table 8. **Module attribution per suite** (LIBERO-Plus, success %), the per-suite version of Tab. 3a.

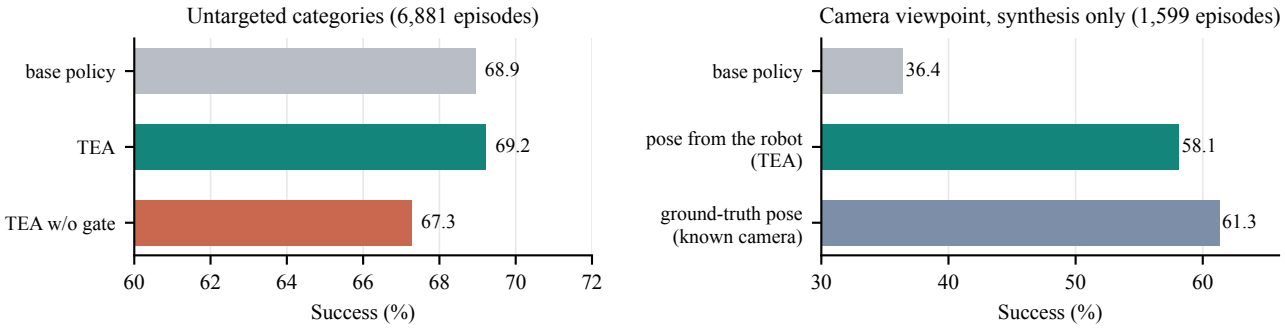

Figure 10. **View Alignment diagnostics** (VLA-Adapter-Pro, identical episodes). Left: all untargeted LIBERO-Plus episodes; without the evidence gate, re-projecting views whose camera has not moved costs 1.9 points against TEA ($p = 8 \times 10^{-8}$), while with the gate TEA matches the base policy. Right: camera-perturbed episodes, synthesis only; giving the ground-truth camera pose adds only 3.2 points over the pose that TEA estimates from the robot itself.

reset state. On RoboTwin, the forward model is fitted on 180 clean demonstrations of six tasks, replayed through the evaluator's command interface (held-out R^2 0.998 / 0.996 for the two arms). Commands are tracked almost exactly (the identity predictor already reaches 0.997), so the gain and drift terms rarely fire there and the effect comes from the start restoration.

Decision rule. All corrections use the same relative decision rule: windowed comparisons of the forward model against (i) the commanded-offset baseline (trust), (ii) its gain-rescaled version one chunk ahead (gain), and (iii) a drift-augmented version that must also beat a fit on the policy's command change (drift). The only constants are the window W (the policy's chunk length) and a sign/order-of-magnitude bound on the gain. Neither is fitted. In full, the gain correction is applied only if the rescaled predic-

Policy	Spatial	Object	Goal	Long	All	p
VLA-Adapter-Pro	98.6	99.4	97.0	94.0	97.2	
+ TEA	98.8	99.0	97.2	95.0	97.5	0.63
OpenVLA-OFT	99.2	98.0	97.0	95.0	97.3	
+ TEA	98.6	96.4	97.6	93.2	96.5	0.075
OpenVLA-OFT+	96.2	98.8	97.2	94.4	96.7	
+ TEA†	96.4	98.2	97.2	93.2	96.2	0.46
$\pi_{0.5}$	98.6	97.8	97.2	92.4	96.5	
+ TEA	99.4	98.2	97.0	92.6	96.8	0.61

Table 9. Clean LIBERO success rates (%), 50 episodes per task (2,000 per policy and arm). p: exact McNemar test on the 2,000 pairs. †OFT+: State Alignment only.

Suite	$W = 4$	$W = 8$ (default)	$W = 16$	Base
Spatial	95.7	97.1	97.1	56.9
Object	89.4	93.5	94.0	38.2
Goal	70.9	75.3	74.6	26.7
Long	78.6	75.3	77.1	33.6

Table 10. Correction window (robot initial state, correction only, Pro).

Category	L1	L2	L3	L4	L5
Robot init.	85.1→96.0	61.0→86.6	36.6→81.8	15.1→78.2	6.6→81.5
Camera	61.3→77.9	61.1→83.1	42.8→71.4	32.5→56.6	5.5→21.6
Language	82.6→82.9	84.3→83.2	86.5→84.9	76.2→78.7	44.2→43.0
Light	87.4→83.8	95.9→91.9	86.9→88.5	65.3→68.6	29.1→35.7
Background	89.7→91.8	85.6→85.2	74.9→77.6	70.6→69.9	43.2→43.8
Noise	83.1→83.1	73.8→72.2	59.7→58.0	41.5→40.7	24.1→25.7
Layout	76.1→75.0	78.8→76.8	81.2→82.1	74.3→75.1	39.4→41.2

Table 13. Success by difficulty level and category (VLA-Adapter-Pro, suites pooled), base → TEA.

Model	L1	L2	L3	L4	L5
OpenVLA-OFT	92.7→93.1	88.0→91.1	76.2→86.6	65.8→79.9	35.4→55.5
OpenVLA-OFT+†	91.9→95.1	89.0→94.8	81.5→93.2	78.1→91.4	64.2→77.1
$\pi_{0.5}$	96.2→95.9	94.3→96.1	90.4→93.4	83.1→88.7	60.6→69.1
VLA-Adapter-Pro	80.4→84.0	75.5→81.3	64.7→77.5	52.5→65.7	24.1→41.5

Table 14. **LIBERO-Plus success rates (%) by perturbation difficulty level**, all categories and suites pooled, base policy → with TEA. †OFT+: State Alignment only.

Δ (pts)	Backgr.	Lang.	Light	Layout	Noise
Pro	+1.0	−0.1	+1.9	0.0	−0.7
$\pi_{0.5}$	+0.5	+0.1	−0.9	+0.1	−0.9
OFT	−0.3	−1.8*	+2.4*	+0.5	+1.7*
OFT+†	−1.1	+0.7	+3.8*	+0.5	+0.5

Table 11. Untargeted LIBERO-Plus categories (pooled over suites), change in points with TEA. *: $p < 0.05$. Over all 6,881 untargeted episodes per policy, the paired 95% bootstrap interval of the change is $[-0.2, +1.1]$ (OFT), $[+0.3, +1.4]$ (OFT+), $[-0.8, +0.4]$ ($\pi_{0.5}$) and $[-0.4, +0.9]$ (Pro): with 95% confidence, TEA costs at most 0.8 points on aggregate for any policy. †OFT+: State Alignment only.

Configuration	Camera	Robot init.	Both
Base policy	24.6	26.2	25.4
+ State Alignment	24.9	30.3	27.6
+ View Alignment	32.6	28.5	30.6
+ TEA	32.9	32.3	32.6

Table 12. Wall-clock time per control step (ms), including policy inference, simulation and registration; VLA-Adapter-Pro, LIBERO-Plus Object, one worker, 15 episodes per category.

Task	Variant	Base	TEA
move near	background 1	50.0	61.7
move near	background 2	65.0	60.0
move near	brighter lighting	48.3	50.0
move near	darker lighting	41.7	46.7
move near	no distractors	43.3	50.0
move near	table texture 1	38.3	40.0
move near	table texture 2	43.3	43.3
pick coke can	background 1	58.7	57.3
pick coke can	background 2	64.0	72.0
pick coke can	brighter lighting	50.7	42.7
pick coke can	darker lighting	48.0	50.7
pick coke can	distractors	56.0	65.3
pick coke can	more distractors	45.3	48.0
pick coke can	table texture 1	56.0	48.0
pick coke can	table texture 2	49.3	52.0

Table 15. SimplerEnv official non-camera variants (GR00T N1.6), success (%), base → View Alignment.

Task	$\pi_{0.5}$ Base	$\pi_{0.5}$ TEA	Fast-WAM Base	Fast-WAM TEA	Task	$\pi_{0.5}$ Base	$\pi_{0.5}$ TEA	Fast-WAM Base	Fast-WAM TEA
adjust_bottle	2/10	3/10	6/10	7/10	place_can_basket	1/10	0/10	4/10	7/10
beat_block_hammer	0/9	3/9	6/7	6/7	place_cans_plasticbox	0/10	0/10	3/10	5/10
blocks_ranking_rgb	2/10	4/10	4/10	6/10	place_container_plate	3/10	4/10	10/10	9/10
blocks_ranking_size	0/10	0/10	9/10	9/10	place_dual_shoes	0/10	1/10	1/10	4/10
click_alarmclock	3/10	6/10	10/10	10/10	place_empty_cup	0/10	4/10	9/10	10/10
click_bell	5/10	7/10	8/10	10/10	place_fan	1/10	3/10	7/10	8/10
dump_bin_bigbin	3/10	4/10	9/10	8/10	place_mouse_pad	1/10	2/10	6/10	6/10
grab_roller	3/10	4/10	9/10	9/10	place_object_basket	2/10	1/10	7/10	6/10
handover_block	0/10	0/10	4/10	5/10	place_object_scale	2/10	1/10	7/10	9/10
handover_mic	0/10	1/10	8/10	9/10	place_object_stand	3/10	4/10	7/10	8/10
hanging_mug	0/10	0/10	5/9	6/9	place_phone_stand	0/10	2/10	5/10	7/10
lift_pot	0/10	0/10	3/10	5/10	place_shoe	0/10	1/10	6/10	6/10
move_can_pot	0/9	0/9	2/6	4/6	press_stapler	2/10	2/10	5/10	6/10
move_pillbottle_pad	0/10	0/10	5/10	7/10	put_bottles_dustbin	0/10	2/10	7/10	7/10
move_playingcard_away	3/10	4/10	9/10	9/10	put_object_cabinet	0/6	0/6	2/7	3/7
move_stapler_pad	0/10	0/10	6/10	6/10	rotate_qrcode	1/10	1/10	6/10	5/10
open_laptop	1/10	4/10	7/10	7/10	scan_object	0/8	0/8	2/8	3/8
open_microwave	3/10	2/10	6/10	3/10	shake_bottle	3/10	4/10	9/10	9/10
pick_diverse_bottles	0/10	0/10	3/10	4/10	shake_bottle_horizontally	4/10	4/10	9/10	9/10
pick_dual_bottles	0/10	0/10	4/10	7/10	stack_blocks_three	0/10	1/10	7/10	9/10
place_a2b_left	0/10	3/10	6/10	7/10	stack_blocks_two	0/10	5/10	7/10	9/10
place_a2b_right	1/10	3/10	7/10	8/10	stack_bowls_three	0/9	5/9	6/10	8/10
place_bread_basket	1/9	1/9	4/10	7/10	stack_bowls_two	4/10	5/10	9/10	10/10
place_bread_skillet	1/7	2/7	3/10	6/10	stamp_seal	2/10	1/10	9/10	6/10
place_burger_fries	1/10	2/10	4/10	5/10	turn_switch	3/10	3/10	6/10	4/10

Table 16. RoboTwin 2.0-Plus, robot-initial-state perturbation: successes per task (paired episodes, common random numbers).

Policy	Perturbation	Spatial	Object	Goal	Long	All 4 suites
OpenVLA-OFT	Action bias	64 → **94** (2.3×10^{-7})	70 → **96** (8.7×10^{-7})	38 → **87** (3.6×10^{-15})	5 → **69** (1.1×10^{-19})	44 → **86** (174/5, 3.9×10^{-45})
	Action uniform	84 → **98** (7.7×10^{-7})	66 → **97** (6.0×10^{-17})	63 → **94** (1.7×10^{-15})	22 → **86** (6.4×10^{-36})	59 → **94** (290/11, 2.0×10^{-71})
	Action noise	43 → 50 (0.09)	22 → **35** (4.2×10^{-5})	20 → **26** (4.3×10^{-3})	5 → 10 (0.08)	22 → **30** (103/40, 1.4×10^{-7})
	External force	36 → 39 (0.25)	19 → 22 (0.26)	38 → 40 (0.73)	22 → 27 (0.06)	29 → **32** (65/39, 0.01)
OpenVLA-OFT+	Action bias	64 → **91** (1.1×10^{-7})	49 → **92** (2.6×10^{-12})	30 → **82** (4.4×10^{-16})	9 → **58** (1.5×10^{-12})	38 → **81** (176/5, 1.0×10^{-45})
	Action uniform	74 → **96** (1.3×10^{-12})	69 → **96** (1.2×10^{-14})	62 → **91** (7.0×10^{-13})	22 → **81** (5.8×10^{-32})	57 → **91** (286/13, 3.9×10^{-68})
	Action noise	42 → 48 (0.21)	22 → **37** (2.0×10^{-4})	28 → 33 (0.24)	12 → 18 (0.11)	26 → **34** (131/72, 4.2×10^{-5})
	External force	37 → 36 (1.00)	22 → **28** (0.02)	36 → 41 (0.09)	22 → 28 (0.07)	29 → **34** (80/46, 3.1×10^{-3})
$\pi_{0.5}$	Action bias	85 → **97** (4.2×10^{-3})	73 → **94** (1.9×10^{-4})	65 → **89** (3.0×10^{-6})	17 → **71** (6.3×10^{-14})	60 → **88** (123/12, 2.3×10^{-24})
	Action uniform	92 → **98** (1.2×10^{-4})	83 → **98** (3.7×10^{-8})	79 → **92** (4.2×10^{-5})	30 → **82** (1.2×10^{-24})	71 → **93** (189/16, 1.1×10^{-38})
	Action noise	72 → 76 (0.43)	43 → **57** (1.3×10^{-3})	50 → 50 (1.00)	18 → **26** (0.04)	46 → **52** (135/83, 5.2×10^{-4})
	External force	60 → 62 (0.66)	34 → 41 (0.06)	54 → 60 (0.10)	32 → **42** (0.01)	45 → **51** (116/68, 5.0×10^{-4})
VLA-Adapter-Pro	Action bias	68 → **97** (3.7×10^{-9})	34 → **86** (6.1×10^{-15})	49 → **85** (1.5×10^{-9})	4 → **67** (3.6×10^{-18})	39 → **84** (184/4, 2.6×10^{-49})
	Action uniform	86 → **98** (4.6×10^{-6})	52 → **92** (7.5×10^{-20})	67 → **90** (3.8×10^{-11})	22 → **78** (1.6×10^{-28})	57 → **89** (279/17, 3.0×10^{-62})
	Action noise	44 → **54** (0.01)	20 → 24 (0.24)	26 → 31 (0.07)	6 → **13** (0.02)	24 → **30** (107/54, 3.6×10^{-5})
	External force	40 → 43 (0.47)	16 → 18 (0.54)	36 → 40 (0.26)	18 → **30** (2.2×10^{-5})	28 → **33** (85/43, 2.6×10^{-4})

Table 17. **RobustVLA perturbations per suite**: base → + State Alignment success (%), 100 paired episodes per suite and perturbation seed (10 per task); two seeds for the random perturbations; in parentheses, the paired McNemar p, and for all suites also the episodes won/lost by State Alignment. Action bias: +0.03 on every action dimension; action uniform: one $\mathcal{U}(-0.04, 0.04)$ offset per episode; action noise: Gaussian, $\sigma = 0.3$; external force: one push per episode of 100–300 N in a random direction for 3–6 steps, as in the released RobustVLA code.

tion $k\,m$, judged one chunk ahead, beats m itself, a fit to the policy's command changes, and the resolution of f (how much better f explains the motion than the commanded offset does, both fed the policy's own commands, $q_t = f(a_{t:t-3}, s_t)$). The drift correction must beat the same two alternatives. When the trust test fails, the statistics are frozen. The gain bound $k \in [0.25, 4]$ is the only constant of the decision rule that is not derived from the demonstrations or from W (Tab. 21).

Idle steps and start restoration. During the evaluator's idle steps the same tests run with a warm-up forward model, fitted on the demonstrations' hold segments, in place of f. At reset, the settling model predicts the pose the nominal arm would settle to; at the policy's first command the statistics restart and the displacement from this pose is restored as described in Sec. 3.1.

Error in what TEA reads	Category	Base	TEA	TEA w/ error
Pose and joint noise, 0.5 mm / 0.25° per step	Robot init.	38.2	84.9	85.6
Pose and joint noise, 2 mm / 1° per step	Robot init.	38.2	84.9	87.7
Joint-reading offset, $\sigma = 1°$	Camera	36.4	57.5	57.2
Joint-reading offset, $\sigma = 3°$	Camera	36.4	57.5	53.7
Focal length assumed by TEA, +5%	Camera	36.4	57.5	58.7
Focal length assumed by TEA, −5%	Camera	36.4	57.5	57.3

Table 18. **TEA under measurement errors** (VLA-Adapter-Pro, LIBERO-Plus, four suites, identical episodes: 1,550 robot-initial-state and 1,599 camera episodes). Only the measurements TEA reads are corrupted; the policy and the simulator are unchanged. Joint offsets are drawn once per episode, as from encoder calibration error, and enter camera registration and robot re-rendering; the focal-length error enters TEA's camera model only.

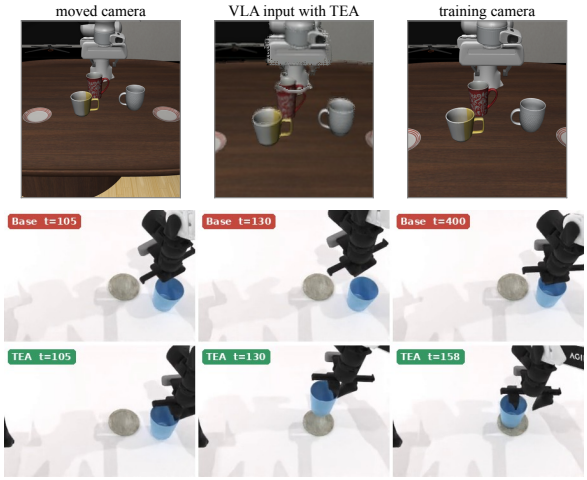

Figure 11. **Top:** the camera episode of Fig. 1 at t=10: the moved view, the view TEA hands to the VLA, and the training camera (rendered for reference only). **Bottom:** paired RoboTwin episode (place_empty_cup, robot initial state, $\pi_{0.5}$).

Embodiment adaptation (RoboTwin). The principle (restore the training start, cancel unexplained drift, act only on evidence) is fixed; the parametrization follows the robot. On RoboTwin's Aloha, commands reach the controller as absolute joint targets ($\pi_{0.5}$ adds its predicted deltas to the observed state), which we convert to increments relative to the previous target of the same chunk, and the groups are the two arms. All training demonstrations start at the same home configuration, so the settled pose is known exactly and the restoration does not depend on f; it is decoupled from the trust test and applied as $\delta/(W\rho)$ per step during the first W corrected steps, where δ is the offset from home.

Window. W is set per benchmark to the number of actions a policy executes per query: 8 on LIBERO (also used for $\pi_{0.5}$, which re-plans every five steps), 32 for $\pi_{0.5}$ and 24 for Fast-WAM on RoboTwin.

View synthesis. Registration runs once, on the first observation after reset, before the idle steps. Candidates come from DISK+LightGlue matches between this image and the training-view reference and its rescaled copies (PnP on the reference depth), and from a 3D pose pool; the best candidates are refined jointly by the render-and-compare score. The robot is rendered at its measured joint angles with the benchmark's robot model (MuJoCo on LIBERO; a renderer built from the URDF with head-joint forward kinematics on SimplerEnv). Each pixel receives a metric depth: the support plane for table pixels, the rendered robot depth for robot pixels, and monocular depth (Depth Anything V2, metric indoor), affinely anchored to the plane and the robot, elsewhere; objects standing on the table are treated as vertical surfaces. Robot pixels visible from $\hat{T}$ are re-projected with their exact depth. Pixels visible only from the training camera are filled from the current image.

What is shared and what is adapted. Table 19 lists, per platform, which module is evaluated, what TEA reads, what is fitted and what follows the robot's interface. The principle (predict the training embodiment, restore its start, cancel unexplained motion, align the view to the training camera, act on relative evidence) is the same everywhere; the parametrization follows the robot's action space and camera mount. None of the platforms requires policy weights, policy gradients or data from the deployment condition; the source-side assets are those available wherever demonstrations were collected.

10. Evaluation protocol

Table 20 summarizes the five benchmarks, the six VLAs and the number of evaluation episodes.

LIBERO-Plus. Camera perturbations rotate the camera in place, change its distance to the scene, or orbit it around the workspace; robot-initial-state perturbations offset the joint configuration at reset. One trial per perturbation variant, seed 7, identical episode lists for base and TEA. Clean

	LIBERO-Plus / LIBERO / RobustVLA	RoboTwin 2.0-Plus	SimplerEnv
Robot, controller	Franka Panda, operational-space	Aloha (dual arm), joint position	Google Robot, as in the benchmark
Policy output	relative end-effector commands	joint deltas added to the state ($\pi_{0.5}$), absolute joint targets (Fast-WAM)	end-effector commands (GR00T N1.6)
Modules evaluated	State and View Alignment (State only on RobustVLA)	State Alignment	View Alignment
What TEA reads	end-effector pose, joints, gripper, executed commands, image	joint readings and drive targets, executed commands	head joints, image
Fitted on the source side	forward, warm-up and settling models on the training demonstrations (warm-up rows rebuilt from the demonstrations' first states)	forward model on clean demonstrations replayed through the command interface; settled pose = home (identical in all demonstrations)	nothing; training camera = mounted camera at default head joints
Adapted to the interface	groups: translation, rotation; restoration decays geometrically and is trust-gated	groups: the two arms; joint increments; restoration constant over the first chunk, not trust-gated, since the home pose is exact	camera candidates from head-joint kinematics and the support plane
Other assets	robot model, camera intrinsics	robot model	robot model (URDF), camera intrinsics
W	8	32 ($\pi_{0.5}$), 24 (Fast-WAM)	–

Table 19. **Per-platform instantiation of TEA.** The same principle and decision rule are used throughout; only the parametrization follows each robot's action space and camera mount.

Benchmark	Simulator, robot	Evaluated perturbations	Tasks	Policies (VLAs)	Episodes
LIBERO-Plus [9]	MuJoCo, Franka	camera, robot init., 5 others (5 levels)	40	OFT, OFT+, $\pi_{0.5}$, Pro	130,728
LIBERO [21]	MuJoCo, Franka	none (clean reference)	40	OFT, OFT+, $\pi_{0.5}$, Pro	21,300
RobustVLA [10]	MuJoCo, Franka	action bias, offset, noise, push (in execution)	40	OFT, OFT+, $\pi_{0.5}$, Pro	34,000
RoboTwin 2.0-Plus [7, 36]	SAPIEN, Aloha	robot init. (both arms); clean	50	$\pi_{0.5}$, Fast-WAM	5,762
SimplerEnv [19]	ManiSkill2, Google	2 alt. cameras; 15 scene variants	2	GR00T N1.6	3,660
5 benchmarks	**3 simulators, 3 robots**	**13 perturbation types**	**92**	**6 VLAs**	**195,450**

Table 20. **Experimental setup.** Episodes include ablations; 125,919 of them are paired comparisons, each run with TEA or an ablation and compared with the same episode under the base policy. Trials per task or variant: Suppl. Sec. 10.

LIBERO uses 50 trials per task.

RobustVLA. We use its four action and environment perturbations on the original LIBERO suites: a constant bias of +0.03 on every action dimension, a uniform offset in $[-0.04, 0.04]$ drawn once per episode, Gaussian action noise ($\sigma = 0.3$), and an external push on the end effector following the released code (one push per episode, 100–300 N in a random direction, 3–6 steps). The perturbation acts on the command sent to the simulator, after TEA, and its random stream is fixed per task and episode, so both arms receive identical perturbations. The camera is nominal, so State Alignment is evaluated alone, with the LIBERO-Plus calibration.

Module assignment. State Alignment needs demonstrations recorded in the evaluated dynamics, and View Alignment serves policies whose behaviour depends on the viewpoint. LIBERO-Plus satisfies both. RoboTwin's demonstrations are simulated and its controller tracks joint targets exactly, so it tests State Alignment's start alignment on a different robot and action space. SimplerEnv's training data are real-robot episodes without joint states, so it tests View Alignment with a head-mounted camera.

RoboTwin 2.0-Plus. The robot-initial-state perturbation applies Gaussian joint offsets ($\sigma = 0.1$ rad, clipped at ± 0.225 rad) to both arms; we also evaluate the clean configuration. Seeds start at $100000 \cdot (1 + s)$ and are filtered by the benchmark's expert check. The expert planner (cuRobo) is not deterministic across runs, so a few seeds pass in only one arm; these are excluded from pairing. Common random numbers: the instruction is chosen with the episode seed, and the flow-matching noise of query i in episode e is seeded with $1000e + i$ in both arms. Under this design, 294 of 294 clean $\pi_{0.5}$ pairs (6 tasks $\times$ 50) execute bit-identical command streams. Observations are rendered only at policy queries ("fast mode"). On eight paired episodes covering clean, robot-state and camera configurations, every chunk, state and command matched the per-step rendering mode bit for bit; no per-render randomness is consumed in these configurations. The gripper "extreme" component of the Plus robot perturbation is overwritten by the environment's gripper-opening step, so only the joint offsets remain.

SimplerEnv. Variant-aggregation base grids (pick coke can: 3 orientations $\times$ 25 positions; move near: 60 episodes), fixed seed, 8-step chunks, 300-step horizon with early termination on success. The camera grid is evaluated three times. The repetitions use the identical episode grid (scenes and environment seeds); they differ only in the sampling of GR00T's action head, which is stochastic, so they measure the policy's own variability. With View Alignment, camera success rises in every repetition: 38.1→43.0%, 36.3→44.8% and 34.4→38.5% (270 paired episodes each). A harness that skips rendering at non-query steps reproduces the original harness exactly (24/24 camera episodes; 30/31 variant episodes, where the remaining one is not reproducible even between two runs of the original harness).

11. Algorithm details

This section specifies both TEA modules exactly as they ran in the final LIBERO-Plus evaluation (State Alignment with its relative decision rule; View Alignment: robot-aware synthesis with current-image fill, evidence gate, 3D pose pool and one registration at reset) and the RoboTwin port of State Alignment. Table 21 lists every constant of TEA.

11.1. State Alignment

Signals. State Alignment wraps the environment's `step`: it runs at *every* control step, including the evaluator's 10 idle steps after reset, which send the fixed hold command $a^{\text{hold}} = (0, 0, 0, 0, 0, 0, -1)$. Let $a_t \in [-1, 1]^7$ be the command the evaluator sends (the policy action after the evaluator's own post-processing), and $c(a) = (u_T a_{1:3},\ u_R a_{4:6},\ a_7)$ its operational-space goal offset, with $u_T = 0.05\,\text{m}$ and $u_R = 0.5\,\text{rad}$ per action unit (the OSC controller's `output_max`). The state before step t is $s_t = (\text{vec}(R_t), p_t, g_t, \sin\theta_t, \cos\theta_t) \in \mathbb{R}^{28}$ (end-effector orientation matrix, position, two finger positions, seven arm joints). The realized motion of step t is $y_t = (p_{t+1} - p_t,\ \text{Log}(R_{t+1}R_t^\top)) \in \mathbb{R}^6$, split into the groups $g \in \{T, R\}$ (dimensions 1:3 and 4:6) with units u_g.

Forward model. f is an MLP ($56 \to 256 \to 256 \to 6$, GELU; standardized inputs, outputs rescaled by the training standard deviation) that predicts y_t from

$$x_t = \big(c(a_t),\ c(a_{t-1}),\ c(a_{t-2}),\ c(a_{t-3}),\ s_t\big) \in \mathbb{R}^{56}, \tag{1}$$

with lags initialized to zero at reset. It is evaluated twice per step on the same state:

$$m_t = f\big(c(\tilde{a}_t), c(\tilde{a}_{t-1:t-3}), s_t\big), \qquad q_t = f\big(c(a^\circ_t), c(a^\circ_{t-1:t-3}), s_t\big), \tag{2}$$

where $\tilde{a}$ is the command actually *executed* (after every correction, with its own lags) and a° is the policy's command before the gain and drift terms (a° already contains the start-restoration offset, see below; its own lags). m_t drives all statistics; q_t only enters the resolution term ϱ. Three networks of this family are used:

- *demo model* f_{demo}: fit on the training transitions of the suite's RLDS demonstrations (`<suite>_no_noops`; episode split: `rng(0)` permutation, first 12% held out).
- *warm-up model* f_{wu}: the same fit on those demo transitions *plus* the evaluator's warm-up rebuilt from the training demonstrations: for every original LIBERO demonstration, its first simulator state is loaded with `env.reset(); env.set_init_state` and the 10 hold steps are simulated and recorded (10 rows per demonstration, same 12% split). f_{wu} predicts the same target y_t and is used only before the policy's first own command.
- *settling model* f_{set}: same inputs, target = the remaining displacement from step t to the end of the 10-step warm-up, $D_t = \big(\sum_{i=t}^{9} y_i^T,\ \text{Log}(\prod_{i=9}^{t} \text{Exp}(y_i^R))\big)$, fit on the same warm-up rows only.

All three: AdamW (lr 10^{-3}, weight decay 10^{-4}), batch 512, 60 epochs, seed 0, MSE on standardized targets. The realization gain is the least-squares slope of realized on commanded offset over the training demo transitions, $\rho_g = \sum_t c(a_t)_g^\top y_t^g \,/\, \sum_t \|c(a_t)_g\|^2$, one scalar per group.

Windowed statistics. With the window W (the policy's executed chunk; $W = 8$) and $\lambda = 1 - 1/W$, every accumulator is a leaky sum $\Sigma[z]_t = \lambda\Sigma[z]_{t-1} + z_t$, initialized to 0, kept per group g on $y = y_t^g$, $m = m_t^g$, $q = q_t^g$, $b = c(a_t^\circ)_g$ (the model-free baseline: the policy's commanded offset) and $\Delta = c(a_t^\circ)_g - c(a_{t-1}^\circ)_g$ (the policy's command change). For a predictor $z \in \{m, b, q\}$ the variation of y left unexplained after its own best scalar scale and vector offset over the window is

$$U(z) = \Big(\Sigma\big[y^\top y\big] - \tfrac{\|\Sigma[y]\|^2}{n}\Big) - \frac{\big(\Sigma[z^\top y] - \Sigma[z]^\top \Sigma[y]\,/n\big)^2}{\Sigma[z^\top z] - \|\Sigma[z]\,\|^2/n}, \qquad n = \Sigma[1]\,, \tag{3}$$

($U = 0$ while $n = 0$; the fraction is dropped when its denominator is $\le 10^{-30}$). The online gain and the drift accumulator are

$$\hat{k} = \text{clip}\Big(\tfrac{\Sigma[y^\top m]}{\Sigma[m^\top m]},\ 0.25,\ 4\Big)\ (\hat{k} = 1 \text{ if } \Sigma\big[m^\top m\big] = 0), \qquad e = \Sigma[k\,m - y]\,, \tag{4}$$

where k is the gain actually applied at that step. Two prediction tests are accumulated on the realized motion, each with the residual it starts from and its projection on the command change:

$$\text{gain:}\quad G_1 = \Sigma\big[\|y - m\|^2\big],\ G_k = \Sigma\big[\|y - \hat{k}_{t-W} m\|^2\big],\ G_r = \Sigma\big[(y-m)^\top\Delta\big],\ G_c = \Sigma\big[\|\Delta\|^2\big], \tag{5}$$

$$\text{drift:}\quad R_0 = \Sigma\big[\|y - km\|^2\big],\ R_1 = \Sigma\big[\|y - km + e/W\|^2\big],\ R_r = \Sigma\big[(y-km)^\top\Delta\big],\ R_c = \Sigma\big[\|\Delta\|^2\big], \tag{6}$$

with k and e taken at decision time. The gain test is therefore judged one chunk ahead: it scores the gain estimated W steps earlier, $\hat{k}_{t-W}$, on the current motion (it starts once $W{+}1$ decisions exist); the drift test is one step ahead. $\Sigma[\cdot]$ of y, m, b, q is updated every step; G_*, R_*, $\Sigma[y^\top m]$, $\Sigma[m^\top m]$ and e are updated only on trusted steps (frozen otherwise).

Decisions. Before each step, per group, from the statistics up to the previous step:

$$\text{trust:} \quad \text{trusted} \iff U(m) \le U(b) \text{ for both } g \in \{T, R\}, \tag{7}$$

$$\text{resolution:} \quad \varrho = \max\big(0,\, U(b) - U(q)\big), \tag{8}$$

$$\text{gain:} \quad k = \hat{k} \text{ if } G_1 - G_k > \max\big(\varrho,\, G_r^2/G_c\big), \text{ else } k = 1, \tag{9}$$

$$\text{drift:} \quad \text{on} \iff R_0 - R_1 > \max\big(\varrho,\, R_r^2/R_c\big), \tag{10}$$

(G_r^2/G_c, R_r^2/R_c are 0 while the denominator is 0). G_r^2/G_c is the error removed by the best scalar fit of the residual on the policy's command change, so motion explained by the command change itself is not evidence; ϱ is what the model currently explains beyond the raw command, so a correction must remove more than that. If trusted, the executed command is

$$\tilde{a}_g = \mathrm{clip}\Big(\frac{a_g^\circ}{k} + \mathbb{1}[\text{drift on}]\,\frac{e}{W\rho_g u_g},\, -1,\, 1\Big), \qquad g \in \{T, R\}, \tag{11}$$

otherwise $\tilde{a} = a^\circ$; the gripper is never modified. $e/(W\rho_g u_g)$ converts the displacement e/W (m or rad) into action units through the demonstrations' per-step realization $\rho_g u_g$. The effective command scale is bounded by $1/k \in [0.25, 4]$ and by the action clip.

Warm-up and start restoration. Onset is the first command that differs from the first command after reset ($a_t \neq a_0$, exact comparison); in LIBERO this is the policy's first own command after the 10 hold steps. Before onset $f = f_{\text{wu}}$ (gain/drift/trust run as above, so a robot-initial-state pull during the hold steps can already be corrected while the nominal settling is explained); from onset on $f = f_{\text{demo}}$. At the first step after reset, from the reset observation and the hold command, f_{set} predicts the nominal settled pose $p^* = p_0 + D^T$, $R^* = \mathrm{Exp}(D^R)R_0$. At onset, all statistics restart from zero (lags and the previous policy command are kept), and, if the first command was repeated at least twice before onset, the remaining displacement

$$\delta_T = p^* - p_{t_0}, \qquad \delta_R = \mathrm{Log}\big(R^* R_{t_0}^\top\big) \tag{12}$$

is measured at the onset step t_0. From t_0 on, at step $t_0{+}j$ ($j = 0, 1, \ldots$; no end), before the decision of Eq. (11):

$$a_g^\circ = \mathrm{clip}\Big(a_g + \lambda^j \frac{\delta_g}{W\rho_g u_g},\, -1,\, 1\Big) \quad \text{if trusted; the factor } \lambda^j \text{ decays also on untrusted steps.} \tag{13}$$

The offsets sum to $\delta_g/(\rho_g u_g)$ action units, *i.e.* the whole displacement at the demonstration realization, with the leaky profile of the drift term ($1 - \lambda^W = 66\%$ after one chunk for $W = 8$). Because the offset is added before f is evaluated, the demo model sees it as commanded motion.

Why the restoration is spread over steps. The displacement δ is added to the policy's commands over several steps rather than in one step, for four reasons. (i) A single step cannot carry it: commands are clipped to $[-1, 1]$, one unit being 5 cm or 0.5 rad of goal offset, while δ is typically several centimetres and more than ten degrees. (ii) The restoration must not override the policy. It is superimposed on the policy's own first commands, which are relative end-effector increments, so each executed step is the policy's intended increment plus a share of δ. Keeping each share at the scale of the demonstrations' commands lets the two motions add up smoothly, whereas a single large offset would dominate the step, jerk the arm and move the next observation away from the training distribution. (iii) It should be largely complete within the policy's first chunk. With the geometric profile, $1 - \lambda^W \approx 66\%$ of δ is restored after $W = 8$ steps, so at its next query the policy plans from a state close to its training start; the remainder fades out over the following steps. (iv) It uses the same law as the drift term, which cancels an accumulated residual e as $e/(W\rho)$ per step, so the restoration needs no constant other than W. Adding the restoration while the policy moves is valid because both the policy's commands and the corrections are increments of the same end-effector goal: each executed step is the motion the policy intends plus a small share of δ, so the correction rides on the policy's motion instead of replacing it, and at the scale of demonstration commands the rotational increments compose

Algorithm 1 State Alignment on LIBERO. Called around every `env.step`.

procedure RESET(o_0) ▷ at `set_init_state`
  all $\Sigma[\cdot] \leftarrow 0$; lags $\leftarrow 0$; onset $\leftarrow$ false; $n_{\text{pre}} \leftarrow 0$; $\delta \leftarrow \varnothing$; $f \leftarrow f_{\text{wu}}$; store s_0
end procedure
procedure CORRECT(a_t) ▷ before each step
  if $t = 0$ **then** $D \leftarrow f_{\text{set}}(c(a_t), 0, 0, 0, s_0)$; $(p^*, R^*) \leftarrow (p_0 + D^T, \text{Exp}(D^R)R_0)$; $a_0^{\text{first}} \leftarrow a_t$
  end if
  if not onset **then**
    if $a_t \neq a_0^{\text{first}}$ **then** onset $\leftarrow$ true ▷ policy's first own command
      all $\Sigma[\cdot] \leftarrow 0$; $f \leftarrow f_{\text{demo}}$
      if $n_{\text{pre}} \geq 2$ **then** $\delta \leftarrow (p^* - p_t,\ \text{Log}(R^* R_t^\top))$; $j \leftarrow 0$
      end if
    else $n_{\text{pre}} \leftarrow n_{\text{pre}} + 1$
    end if
  end if
  $a^\circ \leftarrow a_t$; trusted $\leftarrow U(m) \leq U(b)$ for T and R ▷ Eqs. (3) and (7)
  if $\delta \neq \varnothing$ **then**
    if trusted **then** $a_g^\circ \leftarrow \text{clip}(a_g^\circ + \lambda^j \delta_g / (W \rho_g u_g), -1, 1)$
    end if
    $j \leftarrow j + 1$
  end if
  $\tilde{a} \leftarrow a^\circ$
  for $g \in \{T, R\}$ **do**
    $\hat{k} \leftarrow$ Eq. (4); $\varrho \leftarrow$ Eq. (8); $k \leftarrow$ Eq. (9); drift $\leftarrow$ Eq. (10)
    if trusted **then** $\tilde{a}_g \leftarrow \text{clip}(a_g^\circ / k + \mathbb{1}[\text{drift}]\, e_g / (W \rho_g u_g), -1, 1)$
    end if
  end for
  $m_t \leftarrow f(c(\tilde{a}), \text{exec. lags}, s_t)$; $q_t \leftarrow f(c(a^\circ), \text{policy lags}, s_t)$ ▷ one batched call
  push $\hat{k}$ to a length-$(W{+}1)$ history; $\Delta \leftarrow c(a^\circ) - c(a_{\text{prev}}^\circ)$; update both lag buffers
  return $\tilde{a}$
end procedure
procedure UPDATE(o_{t+1}) ▷ after each step
  $y \leftarrow (p_{t+1} - p_t, \text{Log}(R_{t+1} R_t^\top))$; $\lambda \leftarrow 1 - 1/W$
  for $g \in \{T, R\}$ **do**
    update n and the moments of y, m, b, q
    if trusted at decision **then**
      update R_0, R_1, R_r, R_c (Eq. (6)); if $\hat{k}_{t-W}$ exists, update G_1, G_k, G_r, G_c (Eq. (5))
      $\Sigma[y^\top m]$, $\Sigma[m^\top m]$ updated; $e \leftarrow \lambda e + (km - y)$
    end if
  end for
end procedure

without interference. The policy's first chunk was planned from the uncorrected pose and is shifted slightly while it executes, but the policy queries again after W steps and then plans from a state close to its training start, which is why the profile front-loads the restoration into the first chunk. Empirically, adding the restoration to the warm-up phase raises initial-state success from 79.2% to 84.9% (Tab. 3c), and the same superposition on joint increments carries the gains on RoboTwin. On RoboTwin the same superposition is done on joint increments, with a constant share $\delta/(W\rho)$ over the first chunk.

RoboTwin port. On RoboTwin 2.0 (Aloha-AgileX) the decision rules Eqs. (3) to (11) are unchanged; the parametrization follows the embodiment:

- *Groups and units.* Joint space, groups L, R = the six arm joints of each arm, unit $u_g = 1$ rad; grippers never modified. $W = 32$ (the $\pi_{0.5}$ chunk executed per query).
- *Increments.* pi0.5 outputs absolute joint targets $A_0, \ldots, A_{31}$ per query. With $\bar{s}$ the robot state at the query (`joint_action.vector`, *i.e.* the joints' *drive targets*, not measured positions), the policy command is the increment $c_i^\circ = A_i - A_{i-1}$, $A_{-1} = \bar{s}$ (relative to the previous policy target of the same chunk; the first one relative to the state at the

query). Corrections act on increments and the executed target is accumulated, $A'_i = A'_{i-1} + \tilde{c}_i$, $A'_{-1} = \bar{s}$; with no correction $A' = A$ exactly. A'_i is clipped to $[\min(\ell, A_i), \max(h, A_i)]$ with the joint limits $[\ell, h]$, so the policy's own target always passes.

- *Forward model.* Input $x_i = (A'_i - s_i,$ gripper cmd − gripper state) (executed target minus current drive target), three lags, sin/cos of the measured arm joints, the tracking lag $s_i - q_i^{\text{real}}$ and gripper state (94-d), output the change of the *measured* arm joints (12-d); same network family and training recipe, fit on clean training demonstrations of six tasks replayed through `take_action` in their own seed scenes. q uses $x_i^\circ = (c_i^\circ, \cdot)$ and the baseline is $b = c^\circ$ (without the start offset). ρ_L, ρ_R as above on the replayed rows.
- *Warm-up and settled pose.* RoboTwin's warm-up happens inside `setup_demo` and is not observed; onset is the first corrected step. The settled pose is the first state of the training demonstrations, which is identical for all of them (standard deviation 0; equal to the embodiment's home state), so f_{set} is that constant and $\delta = s^{\text{home}} - \bar{s}_0$ is measured on drive targets.
- *Undo profile.* $\tilde{c}_g \mathrel{+}= \delta_g / (W \rho_g u_g)$ (constant, no decay) on each of the first W corrected steps (the first chunk), then $\delta \leftarrow \varnothing$; not gated by the trust test. Gain and drift remain trust-gated, and the offset is subject to the gain division and the joint-limit clip like the policy increment.

11.2. View Alignment

Calibration from training demonstrations (offline, once per suite or task). Intrinsics K are known (`fovy`= 45°, 224×224 policy image). The reference image I^{ref} of a task is the pixel-wise median of the first frames of its training demonstrations. The training camera $T_0 = (R_0, t_0)$ (robot-base frame), the start joints q^{home} (median joint state at $t = 0$), a per-channel affine colour response κ (robot render → image) and the support-plane height z_s (0.5th percentile of the lowest fingertip point over training frames) are estimated from training demonstrations only by render-and-compare of a robot-only MuJoCo model (robot + gripper + mount meshes, no scene). The reference depth D^{ref} is the robot render depth at (T_0, q^{home}) on the robot mask and, elsewhere, monocular depth anchored as in Eq. (15) to the plane depth (on reference pixels of the training-view plane) and the robot depth; values outside $(0, 6]$ m are invalid. It is computed once per reference and cached.

When registration runs. Exactly once per episode, on the external image I_0 of the observation returned by `set_init_state` (before the 10 hold steps and before any State Alignment action), with the proprioceptive joints (q_0, g_0) of that observation. The decision and the pose are kept for the whole episode; queries never re-register. The task's reference is the training task whose instruction prefixes the episode instruction (otherwise, the reference with the most homography inliers).

Pose candidates. I_0 is matched (DISK, 2048 keypoints, images upsampled 2×, LightGlue) against the reference and 20 shrunk copies of it (scales $\{0.7, 0.55, 0.45, 0.38, 0.32\}\times$ vertical centres $\{112, 60, 10, -40\}$ px; features cached per reference), giving up to 21 match sets. A set is kept if it has ≥ 12 matches whose reference point has valid D^{ref} and a MAGSAC homography exists (3 px, 5000 iterations). For each kept set, the reference points are lifted to 3D with D^{ref} and T_0 and the current camera T_j is solved by EPnP-RANSAC (3 px, 3000 iterations, ≥ 8 inliers) followed by Levenberg–Marquardt refinement. The candidate with the highest robot score S (Eq. (14)) is selected; it is then refined by Levenberg–Marquardt on all matches of all sets that reproject within 3 px under it (if ≥ 8) and rescored, giving $\hat{T}$ with score $S(\hat{T})$.

Robot score. With the robot rendered at camera T and the current joints, mask M_T and colour $\kappa(\text{RGB}_T)$,

$$S(T) = \text{ZNCC}_{\Omega_T}\big(\text{gray}(\kappa(\text{RGB}_T)),\ \text{gray}(I)\big), \qquad \Omega_T = \text{dilate}_{7\times7}(M_T), \tag{14}$$

i.e. the zero-mean normalized cross-correlation over the robot and a 3 px ring around it ($S = -1$ if $|M_T| < 30$ or $|\Omega_T| < 50$).

Evidence gate. Synthesis is enabled iff a candidate exists and $S(\hat{T}) > S(T_0)$, both evaluated on the same registration frame I_0 with the same joints. Otherwise the raw image is passed for the whole episode.

Per-query synthesis. At every policy query, with the current image I and joints (q, g): *(i) depth.* Plane depth D_Π of $z = z_s$ seen from $\hat{T}$; robot render at $\hat{T}$ (mask M, depth D_M); monocular depth d (Depth Anything V2 metric-indoor Small, input 518). The monocular depth is anchored by an inverse-depth affine fit

$$\tfrac{1}{D_{\text{anc}}} = \alpha \tfrac{1}{d} + \beta, \qquad (\alpha, \beta) = \arg\min \sum_{u \in P} \Big(\alpha \tfrac{1}{d_u} + \beta - \tfrac{1}{D_{\Pi,u}}\Big)^2 + \sum_{u \in M} \Big(\alpha \tfrac{1}{d_u} + \beta - \tfrac{1}{D_{M,u}}\Big)^2, \tag{15}$$

Algorithm 2 View Alignment (robot-aware synthesis, evidence gate, pose pool, registration at reset).

procedure PRIME(I_0, q_0, g_0) ▷ once, at `set_init_state`
  pick reference I^{ref} by instruction; $D^{\text{ref}} \leftarrow$ cached reference depth
  $C \leftarrow \varnothing$
  for $(\sigma, y_c) \in \{(1, 112)\} \cup \{0.7, 0.55, 0.45, 0.38, 0.32\} \times \{112, 60, 10, -40\}$ **do**
    match I_0 to I^{ref} shrunk by σ about $(112, y_c)$; map matches back to I^{ref}
    keep matches with valid D^{ref}; **skip** if < 12 or no MAGSAC homography
    lift to 3D with D^{ref}, T_0; $T_j \leftarrow$ EPnP-RANSAC + LM; **skip** if < 8 inliers
    $C \leftarrow C \cup \{(T_j, S(T_j))\}$ ▷ Eq. (14), robot at (q_0, g_0)
  end for
  if $C = \varnothing$ **then** $\hat{T} \leftarrow \varnothing$; **return**
  end if
  $\hat{T} \leftarrow \arg\max_C S$; refine by LM on all matches within 3 px of $\hat{T}$ (if ≥ 8); rescore
  if $S(\hat{T}) \leq S(T_0)$ **then** $\hat{T} \leftarrow \varnothing$
  end if ▷ evidence gate, same frame
end procedure
procedure QUERY(I, q, g) ▷ every policy query
  if $\hat{T} = \varnothing$ **then return** I
  end if
  $d \leftarrow \text{DA2}(I)$ **in parallel with** robot renders at $\hat{T}$ (mask, depth) and at T_0 (RGB, depth, mask)
  $D \leftarrow$ plane / robot / anchored-mono / billboard composition (Eq. (15))
  forward-splat into T_0 (z-buffer), close holes, inverse map + bilinear sampling, occlusion test
  paint unseen visible robot pixels from $\kappa(\text{RGB}_{T_0})$; inpaint remaining unseen pixels
  return synthesized image
end procedure

over plane pixels P (ray hits the plane, outside the 5×5-dilated robot mask) and robot pixels, in two passes, the second keeping only plane pixels with $|D_{\text{anc}} - D_\Pi|/D_\Pi < 0.04$. Pixels whose anchored 3D point lies more than 0.015 m above the plane (morphologically opened) and pixels whose ray misses the plane take D_{anc}; each above-plane connected component that does not touch the robot and has ≥ 6 pixels is replaced, column by column, by a vertical plane through its floor contact facing the camera (billboard); robot pixels take D_M; all remaining pixels take D_Π. *(ii) re-projection.* Pixels are back-projected with $\hat{T}$ and forward-splatted into T_0 with a z-buffer, $\mathbf{u}_0 = K\pi\big(T_0\hat{T}^{-1}D(\mathbf{u})K^{-1}\tilde{\mathbf{u}}\big)$; holes of the target depth are closed by four 3×3 min-filter passes; each target pixel is then mapped back into I and sampled bilinearly; it counts as seen if it lands inside I and is not occluded ($z \leq 1.05\, D(\mathbf{u})$); the seen mask is eroded 3×3. *(iii) robot.* The robot is rendered at T_0 with (q, g); its visible pixels that are not seen are painted with $\kappa(\text{RGB}_{T_0})$ (seen robot pixels keep the re-projected current colour, which used the exact robot depth). *(iv) holes.* All other unseen pixels are inpainted from the synthesized pixels (Telea, radius 3); nothing is copied from training images.

Cost. Per episode: one registration (one DISK extraction on I_0; up to 21 LightGlue matchings, MAGSAC homographies and PnP solves; up to 23 robot renders for scoring, refit and T_0); reference features of the shrunk copies and D^{ref} are cached per task across episodes. Per query: one monocular-depth inference, two robot renders, a two-pass 2-parameter least-squares fit, the billboard pass, one splat and one remap, and inpainting. State Alignment: one batched two-row MLP evaluation per control step (plus one settling-model evaluation per episode).

Table 21. Constants of TEA. *Design*: fixed by the interface, the policy's execution structure or the robot model; *data*: derived from training demonstrations only (which ones in brackets); *set once*: chosen a priori, identical for every suite, policy and benchmark, never adjusted on evaluation episodes.

Constant	Value	Status	Origin / remark
State Alignment (LIBERO)			
Window W, $\lambda = 1 - 1/W$	8, 0.875	design	executed chunk of VLA-Adapter and OpenVLA-OFT; also used for pi0.5, which executes 5 actions per query
Gain bound	$[0.25, 4]$	set once	order-of-magnitude bound on $\hat{k}$, not fitted
Units u_T, u_R	0.05 m, 0.5 rad	design	OSC controller `output_max`
Action clip	$[-1, 1]$	design	action space
ρ_T/ρ_R	Spatial .229/.186, Object .243/.209, Goal .196/.192, Long .233/.216	data	LS slope realized vs. commanded offset (suite's training demos)
f_{demo}, f_{wu}, f_{set} weights	–	data	suite's training demos; warm-up rows rebuilt from the demos' first simulator states
Lags; network; training	3; 2×256 GELU; AdamW 10^{-3}, wd 10^{-4}, batch 512, 60 ep., seed 0, 12% hold-out	set once	identical for all three networks, all suites (and RoboTwin)
Warm-up length, hold command	10, $(0, 0, 0, 0, 0, 0, -1)$	design	evaluator protocol (`num_steps_wait`); only used to generate warm-up rows
Undo condition	$n_{\text{pre}} \geq 2$	design	a warm-up regime exists (first command repeated)
Numerical guard	10^{-30}	design	denominator of Eq. (3)
State Alignment (RoboTwin)			
W	32	design	$\pi_{0.5}$ actions executed per query
ρ_L/ρ_R	.996/.995	data	replayed clean demos of 6 training tasks
Settled pose	home state	data	first state of all training demos (std 0)
Unit; joint limits	1 rad; robot model	design	absolute-qpos interface
Undo profile; undo trust gate	constant over first chunk; off	set once	embodiment adaptation
View Alignment			
Image size, `fovy`	224, 45°	design	policy input; known intrinsics
I^{ref}	per task	data	pixel median of training first frames
T_0, q^{home}, κ, z_s	per suite/task	data	render-and-compare on training frames; median t=0 joints; colour LS; 0.5th pct. lowest fingertip
Shrunk copies	$\{0.7, .55, .45, .38, .32\} \times \{112, 60, 10, -40\}$ px	set once	fixed a priori (zoom-out hypotheses)
Matcher	DISK 2048 kp, 2× upsampling, LightGlue	set once	pretrained weights
Match / homography	≥ 12; MAGSAC 3 px, 5000 it., 0.999	set once	
PnP; pooled refit	EPnP-RANSAC 3 px, 3000 it., ≥ 8; LM, 3 px, ≥ 8	set once	
Robot score	ring 7×7; $\lvert M\rvert \geq 30$, $\lvert\Omega\rvert \geq 50$ px	set once	
Plane tolerance; passes	0.04 (relative); 2	set once	anchoring inliers
Height above plane	0.015 m	set once	above-plane pixels
Masks	robot exclusion 5×5; billboard ≥ 6 px, touch 5×5; ref. floor < 4 m, 9×9; ref. depth ≤ 6 m	set once	
Re-projection	occlusion 1.05; 4 hole passes 3×3; erosion 3×3; Telea radius 3	set once	
Monocular depth	DA2 metric-indoor Small, input 518, `max_depth` 20	design	pretrained checkpoint config